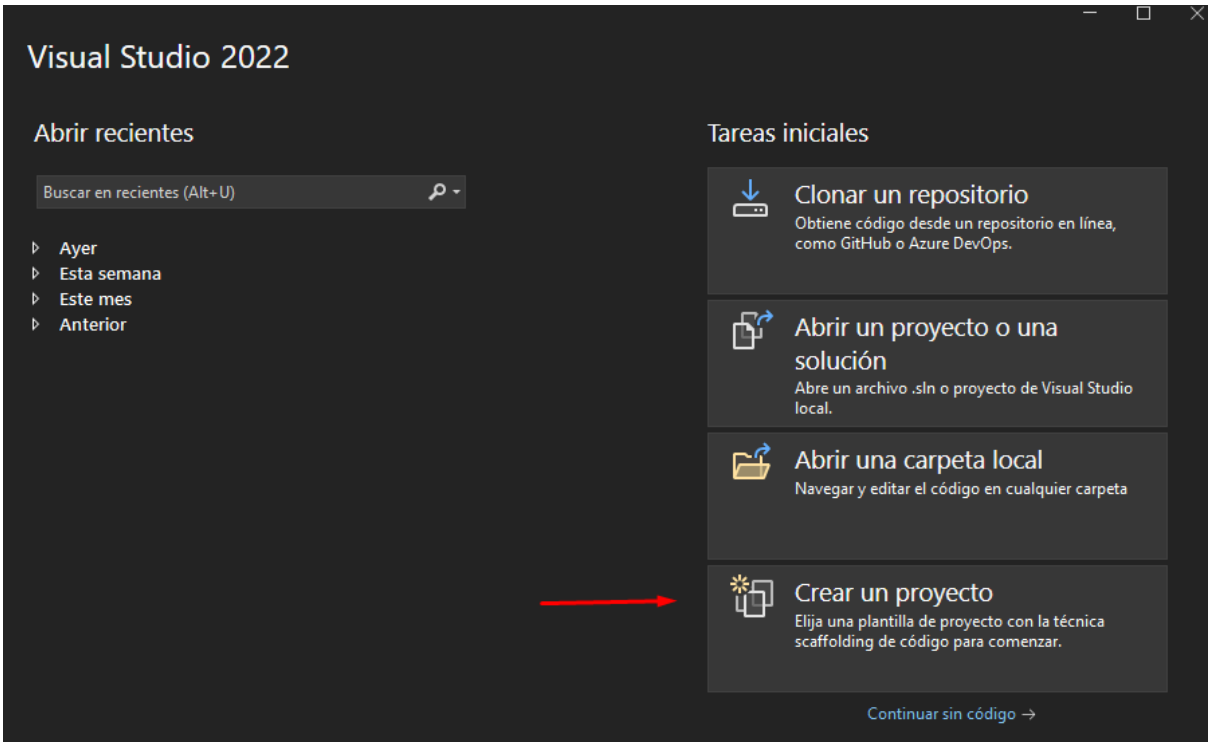
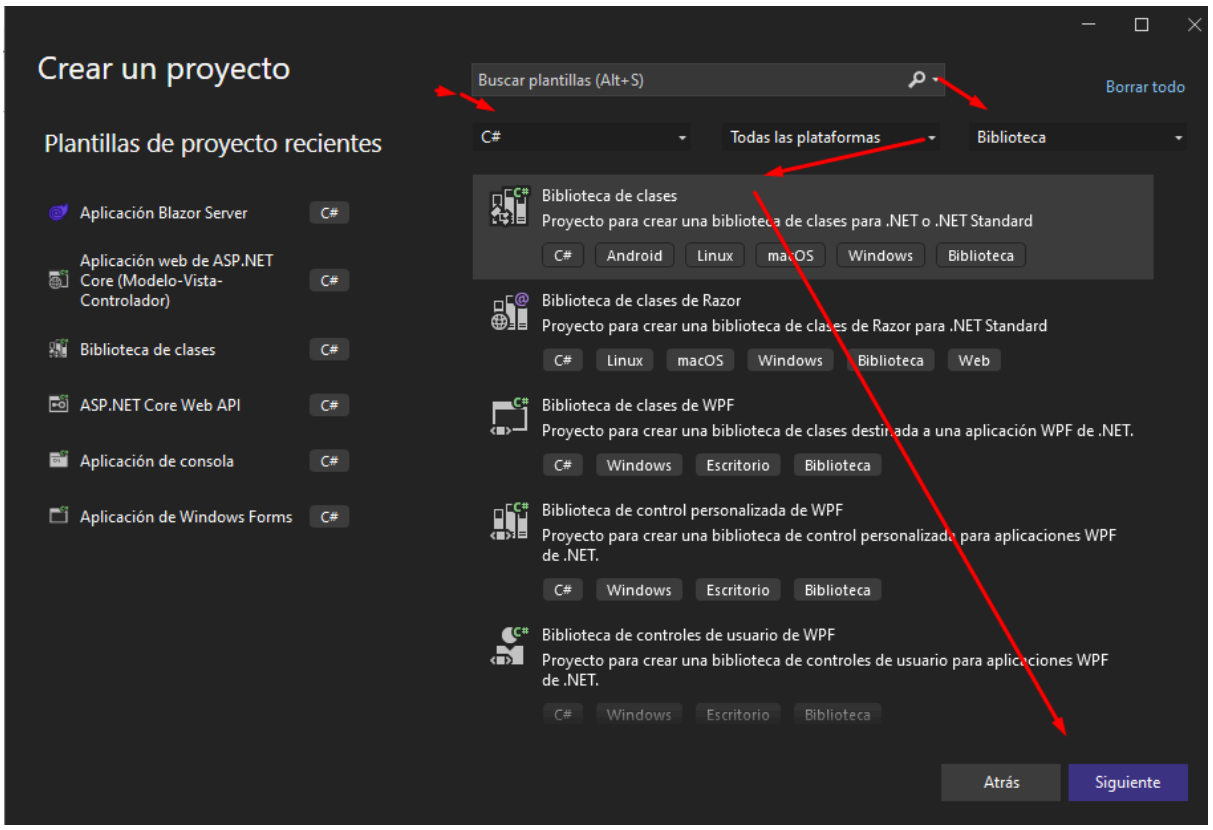
Pruebas unitarias con MSTest, xUnit

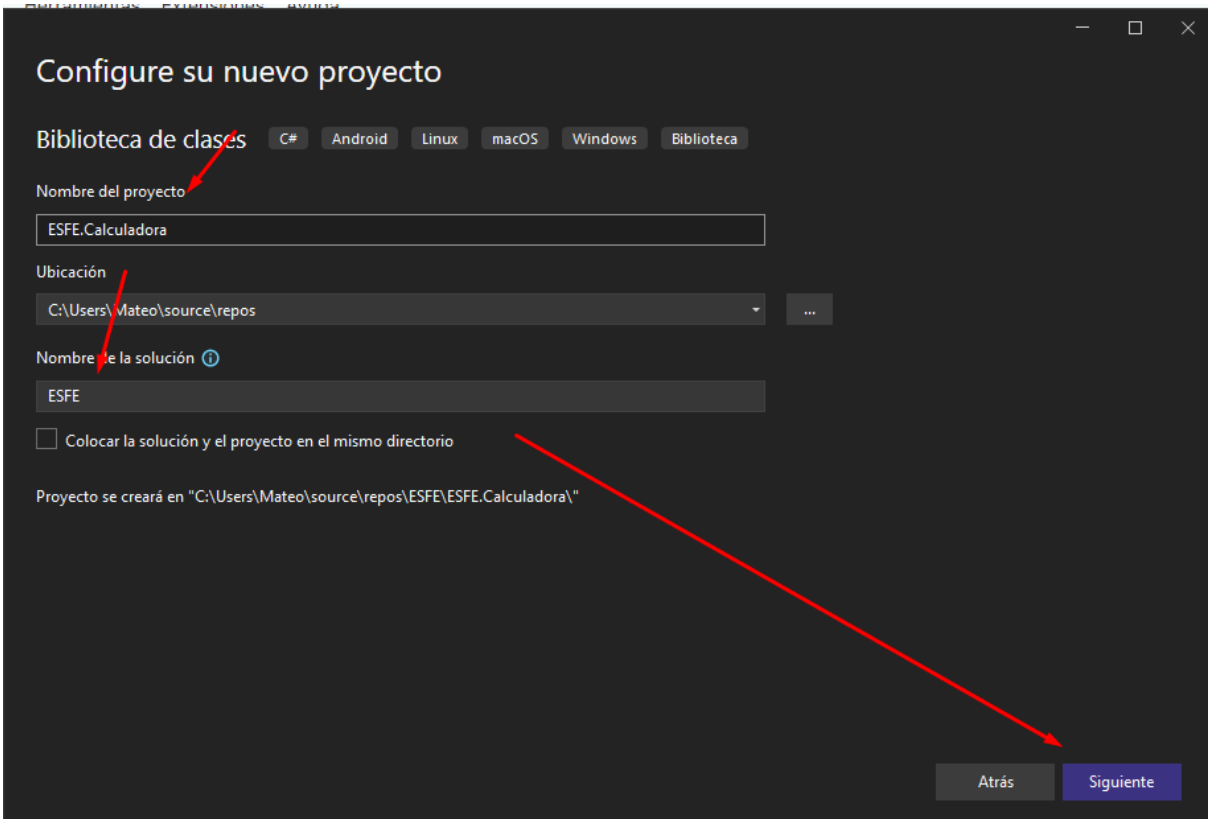
Para crear pruebas unitarias con Visual Studio 2022, te recomendamos que antes de comenzar, leas los siguientes artículos:
https://learn.microsoft.com/es-es/visualstudio/test/unit-test-basics?view=vs-2022

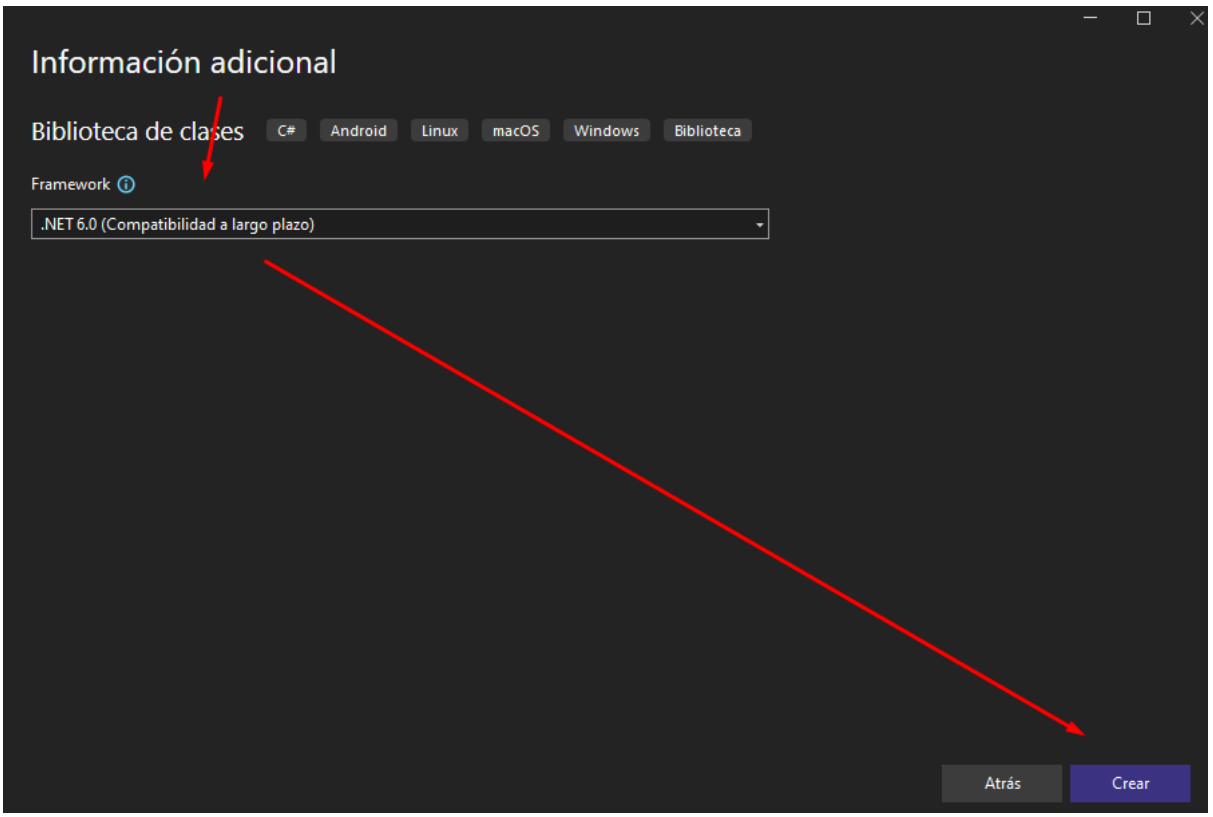
https://learn.microsoft.com/es-es/visualstudio/test/how-to-create-a-data-driven-unit-test?view=vs-2022

Vamos a crear un proyecto de biblioteca de clases (class library) para que podamos realizar pruebas unitarias en ese proyecto.

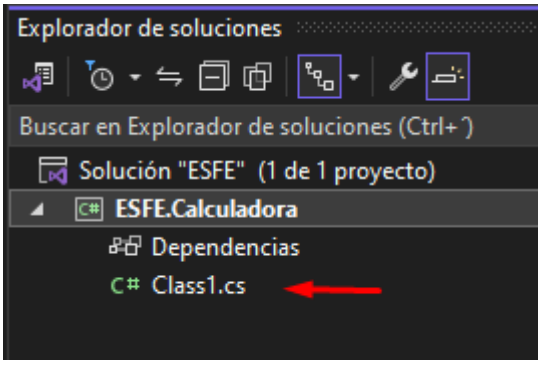


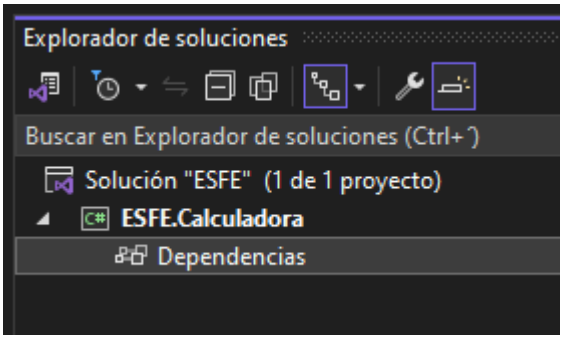

Crear un proyecto
Buscar plantillas (Alt+S)
Borrar todo
Plantillas de proyecto recientes
C#
Todas las plataformas
Biblioteca
Aplicación Blazor Server C#
Aplicación web de ASP.NET Core (Modelo-Vista-Controlador) C#
Biblioteca de clases C#
ASP.NET Core Web API C#
Aplicación de consola C#
Aplicación de Windows Forms C#
Biblioteca de clases
Proyecto para crear una biblioteca de clases para .NET o .NET Standard
C# Android Linux macOS Windows Biblioteca
Biblioteca de clases de Razor
Proyecto para crear una biblioteca de clases de Razor para .NET Standard
C# Linux macOS Windows Biblioteca Web
Biblioteca de clases de WPF
Proyecto para crear una biblioteca de clases destinada a una aplicación WPF de .NET.
C# Windows Escritorio Biblioteca
Biblioteca de control personalizada de WPF
Proyecto para crear una biblioteca de control personalizada para aplicaciones WPF de .NET.
C# Windows Escritorio Biblioteca
Biblioteca de controles de usuario de WPF
Proyecto para crear una biblioteca de controles de usuario para aplicaciones WPF de .NET.
C# Windows Escritorio Biblioteca
Atrás
Siguiente


Configure su nuevo proyecto
Biblioteca de clases
C# Android Linux macOS Windows Biblioteca
Nombre del proyecto
ESFE.Calculadora
Ubicación
C:\Users\Mateo\source\repos
...
Nombre de la solución
ESFE
Colocar la solución y el proyecto en el mismo directorio
Proyecto se creará en "C:\Users\Mateo\source\repos\ESFE\ESFE.Calculadora\"
Atrás
Siguiente

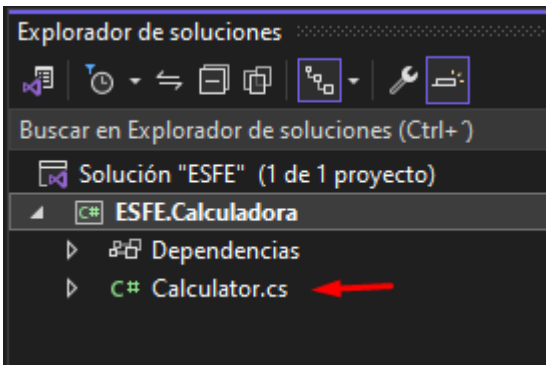


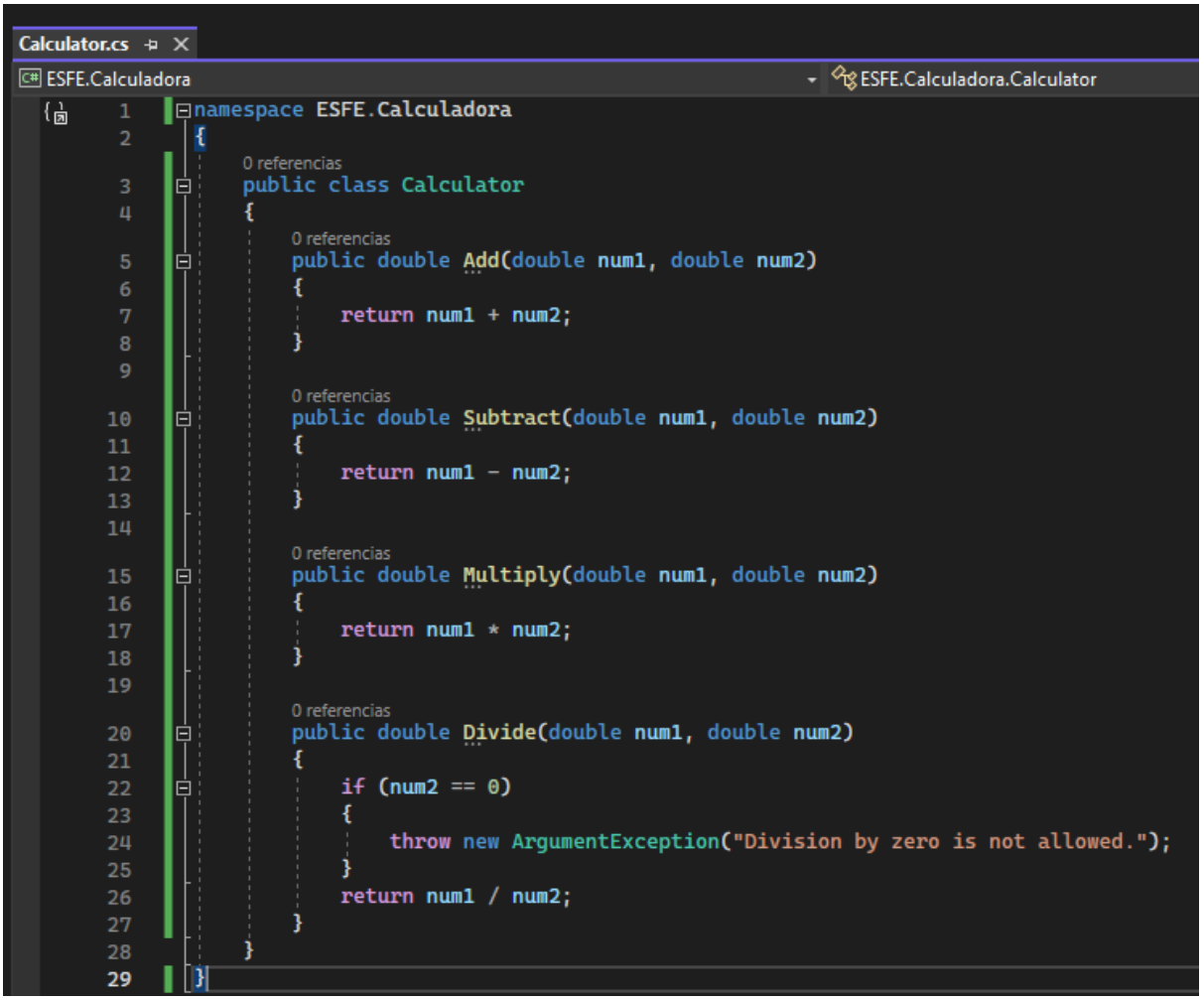
Eliminamos la clase por defecto.



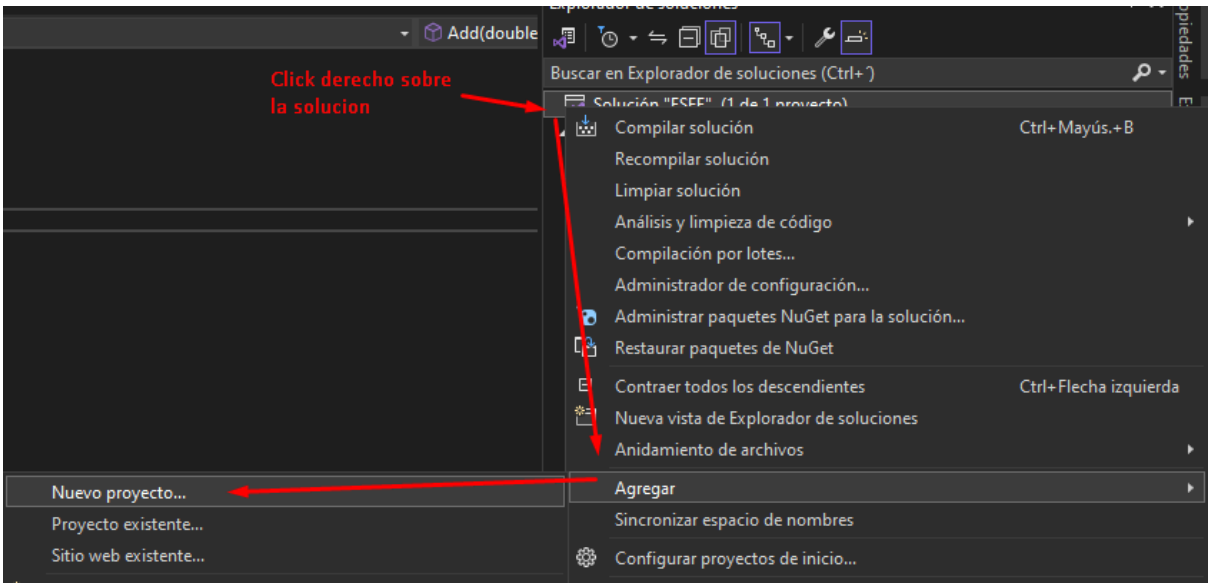


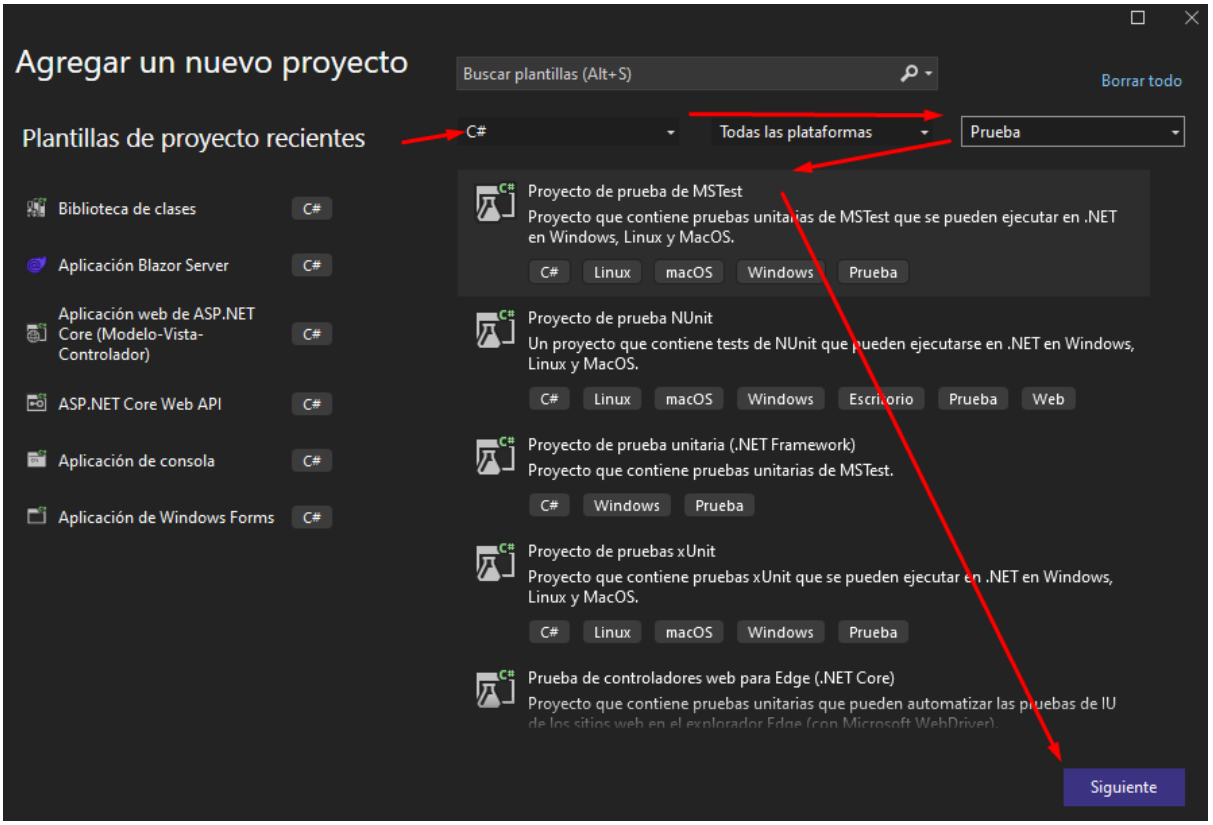
Ahora, crearemos una clase llamada 'Calculator' y modificaremos su código por defecto.

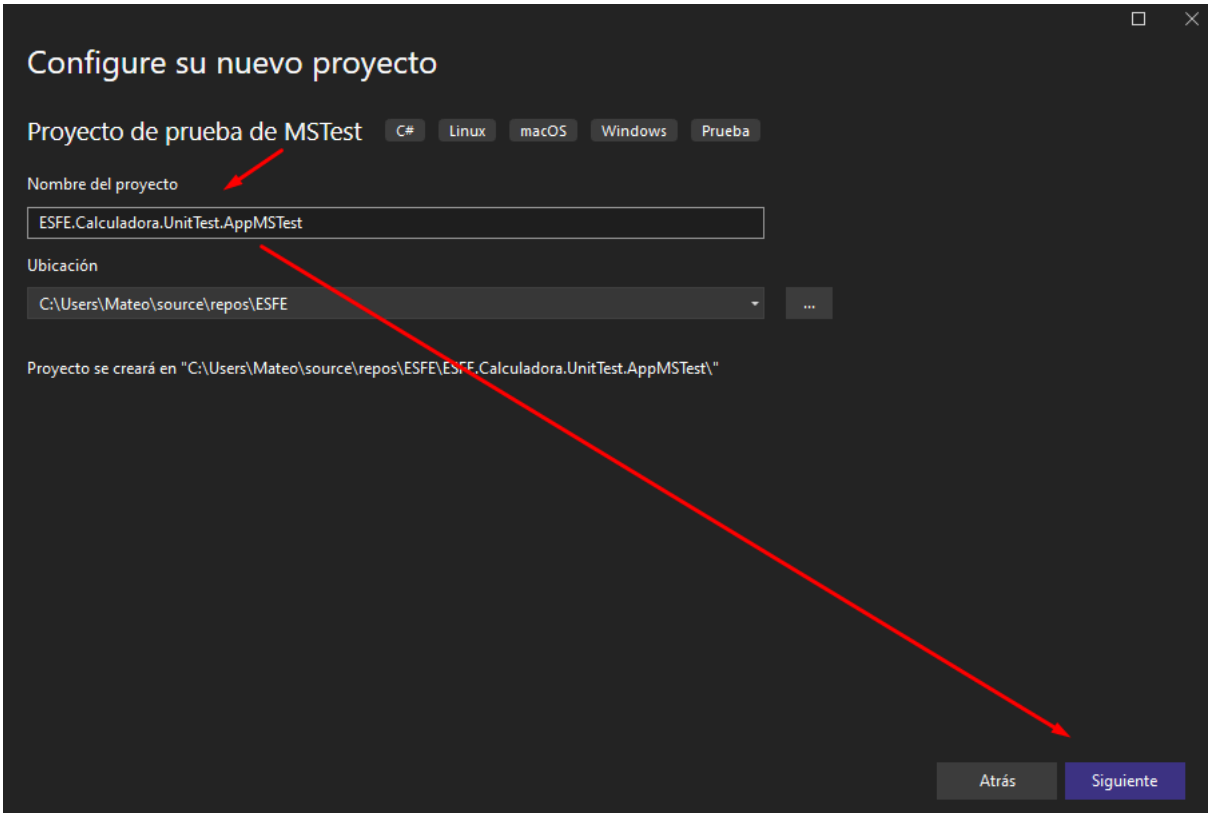


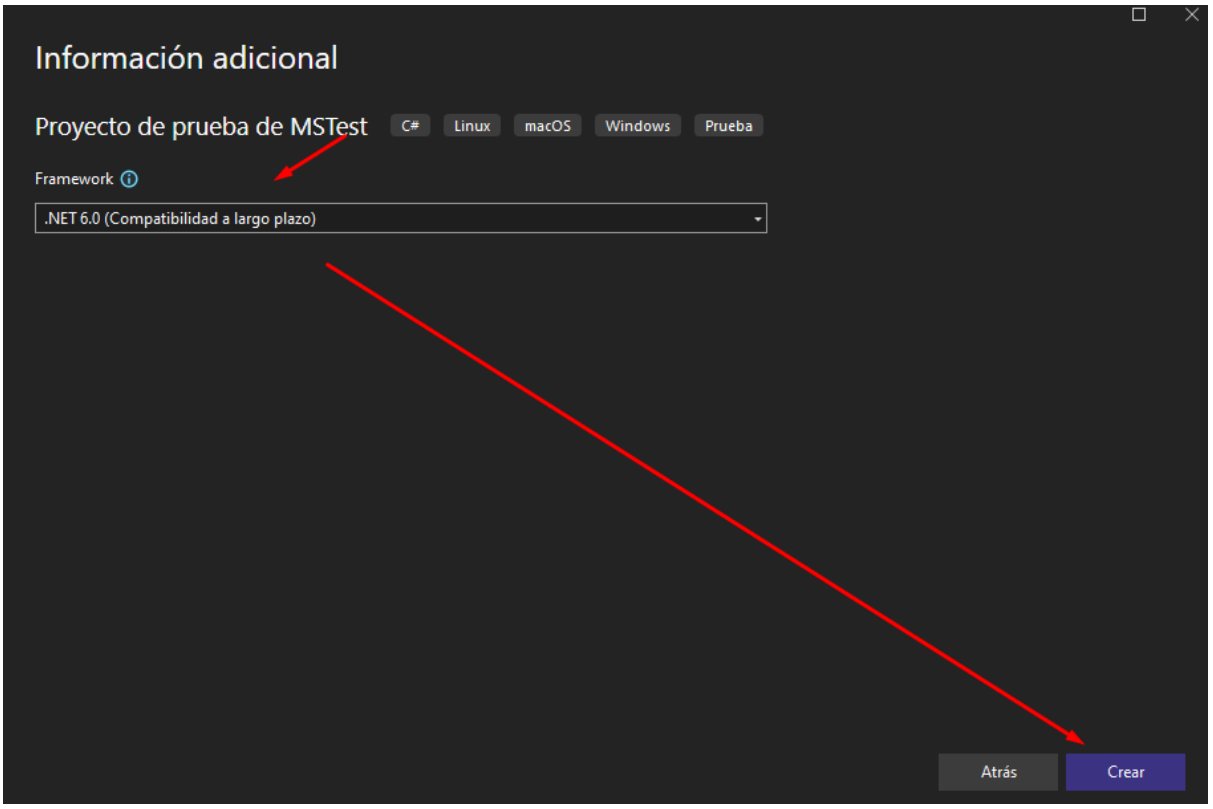


Ahora, procederemos a crear un proyecto para pruebas unitarias de tipo MSTest.

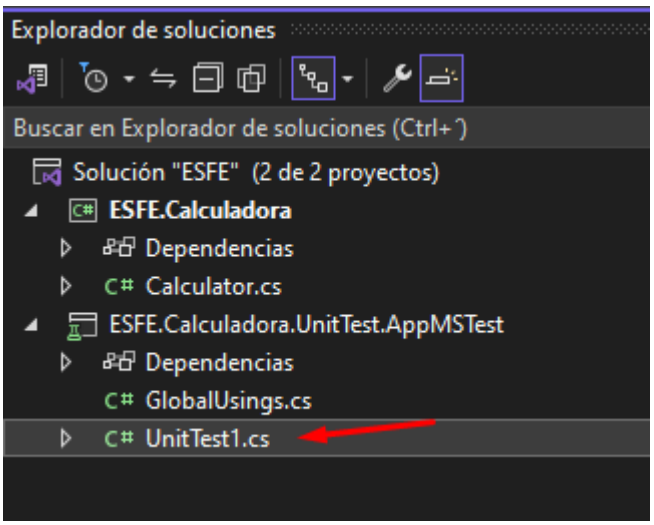

Add(double
Click derecho sobre
la solucion
Buscar en Explorador de soluciones (Ctrl+ˀ)
Compilar solución Ctrl+Mayús.+B
Recompilar solución
Limpiar solución
Análisis y limpieza de código
Compilación por lotes...
Administrador de configuración...
Administrar paquetes NuGet para la solución...
Restaurar paquetes de NuGet
Contraer todos los descendientes Ctrl+Flecha izquierda
Nueva vista de Explorador de soluciones
Anidamiento de archivos
Nuevo proyecto...
Agregar
Proyecto existente...
Sincronizar espacio de nombres
Sitio web existente...
Configurar proyectos de inicio...


Agregar un nuevo proyecto
Buscar plantillas (Alt+S)
Borrar todo
Plantillas de proyecto recientes
C#
Todas las plataformas
Prueba
Biblioteca de clases C#
Aplicación Blazor Server C#
Aplicación web de ASP.NET Core (Modelo-Vista-Controlador) C#
ASP.NET Core Web API C#
Aplicación de consola C#
Aplicación de Windows Forms C#
Proyecto de prueba de MSTest
Proyecto que contiene pruebas unitarias de MSTest que se pueden ejecutar en .NET en Windows, Linux y MacOS.
C# Linux macOS Windows Prueba
Proyecto de prueba NUnit
Un proyecto que contiene tests de NUnit que pueden ejecutarse en .NET en Windows, Linux y MacOS.
C# Linux macOS Windows Escritorio Prueba Web
Proyecto de prueba unitaria (.NET Framework)
Proyecto que contiene pruebas unitarias de MSTest.
C# Windows Prueba
Proyecto de pruebas xUnit
Proyecto que contiene pruebas xUnit que se pueden ejecutar en .NET en Windows, Linux y MacOS.
C# Linux macOS Windows Prueba
Prueba de controladores web para Edge (.NET Core)
Proyecto que contiene pruebas unitarias que pueden automatizar las pruebas de IU
Siguiente

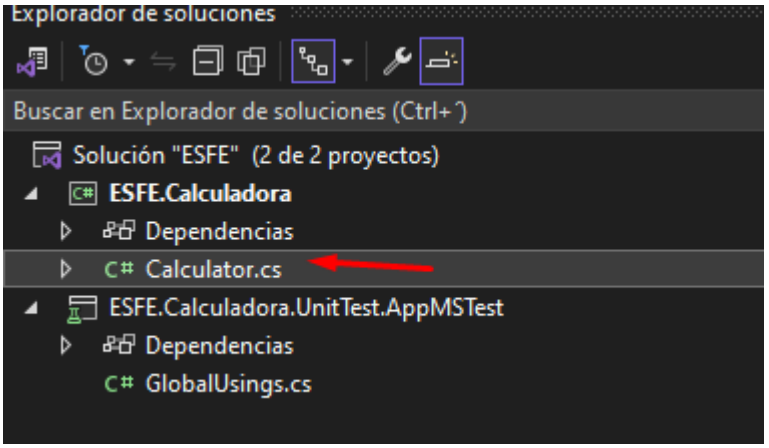


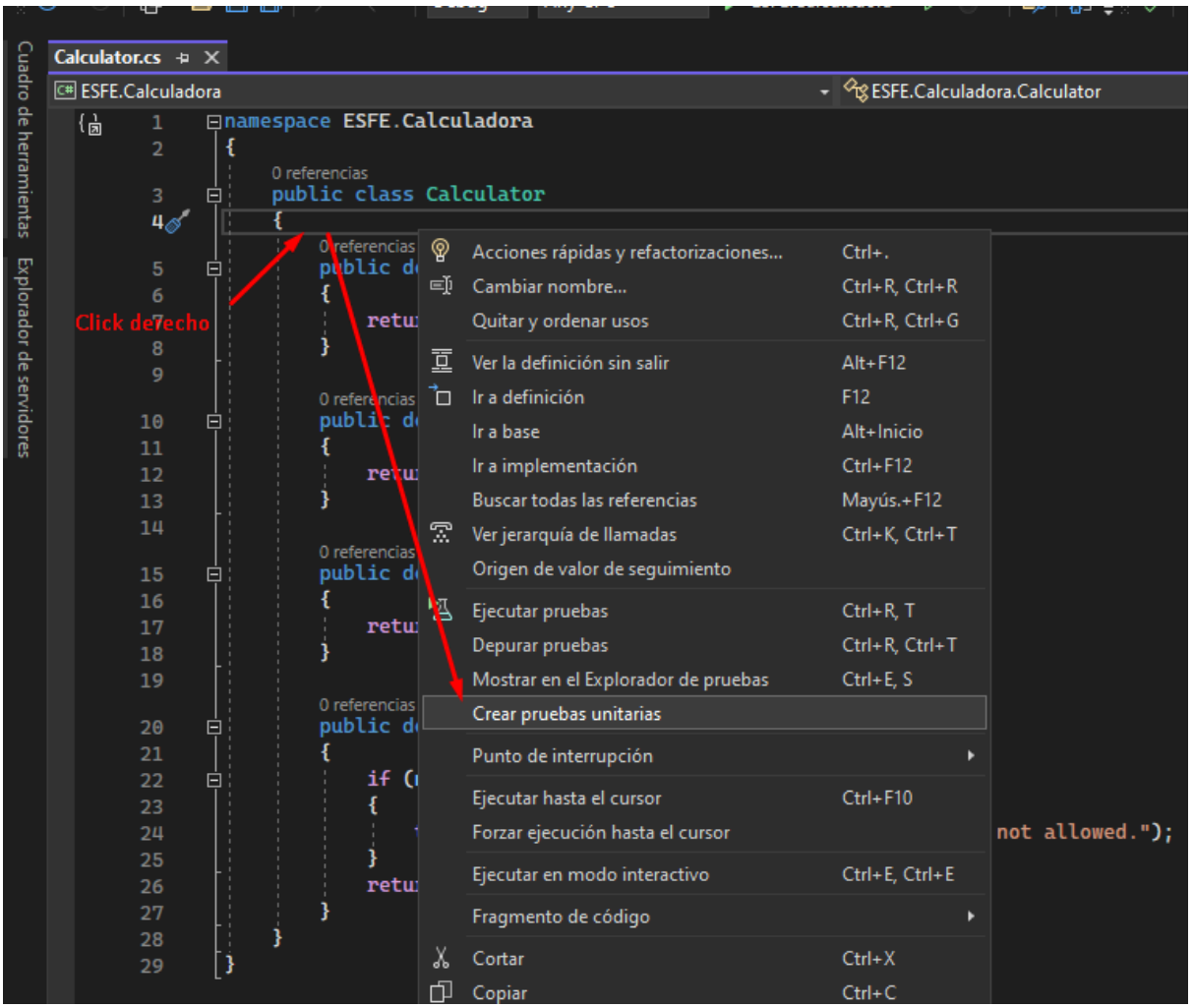


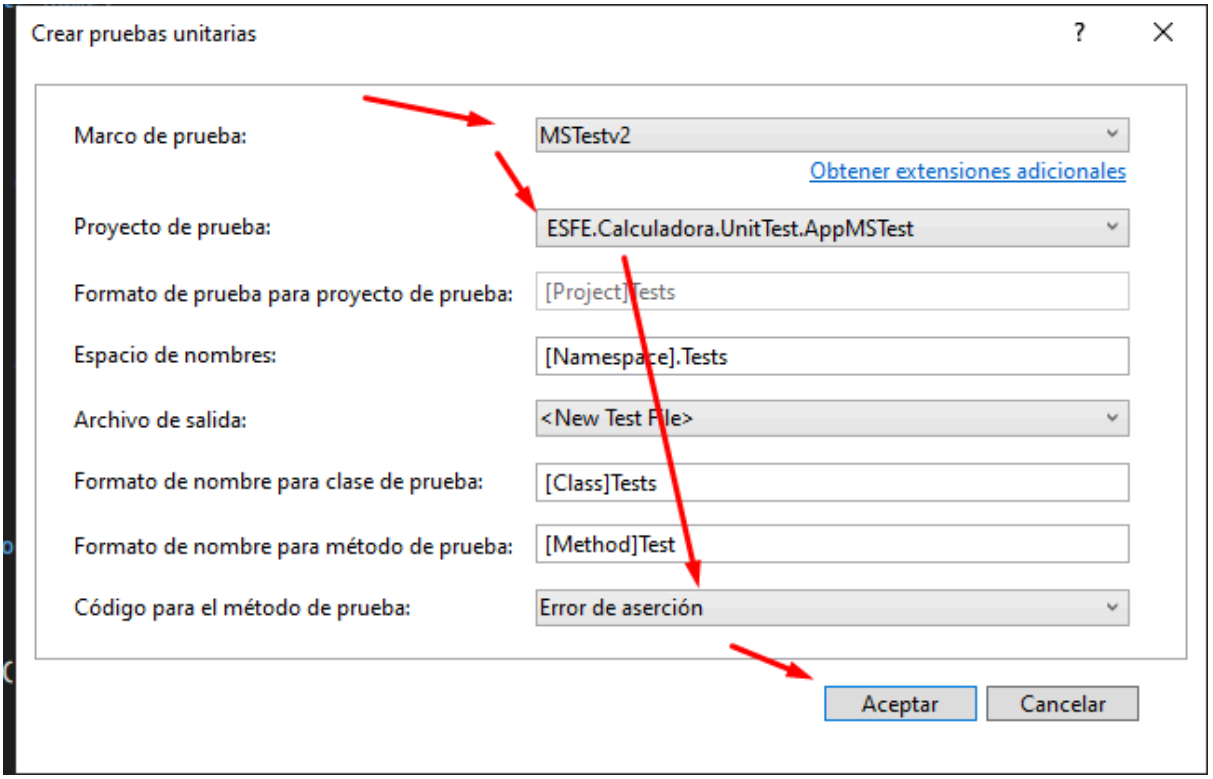
Vamos a eliminar la clase que se creó por defecto.

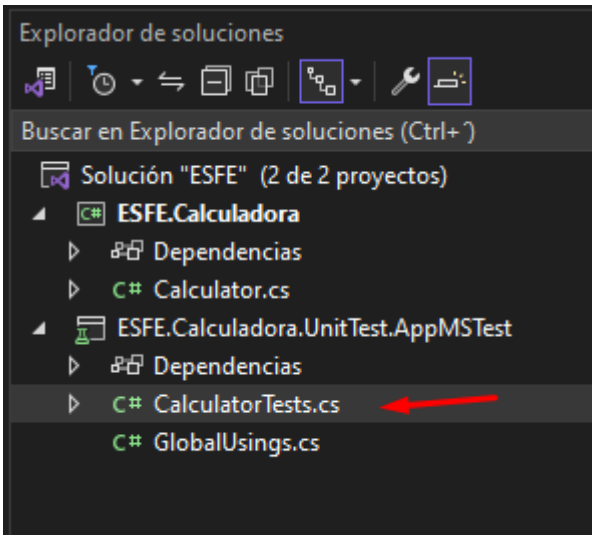


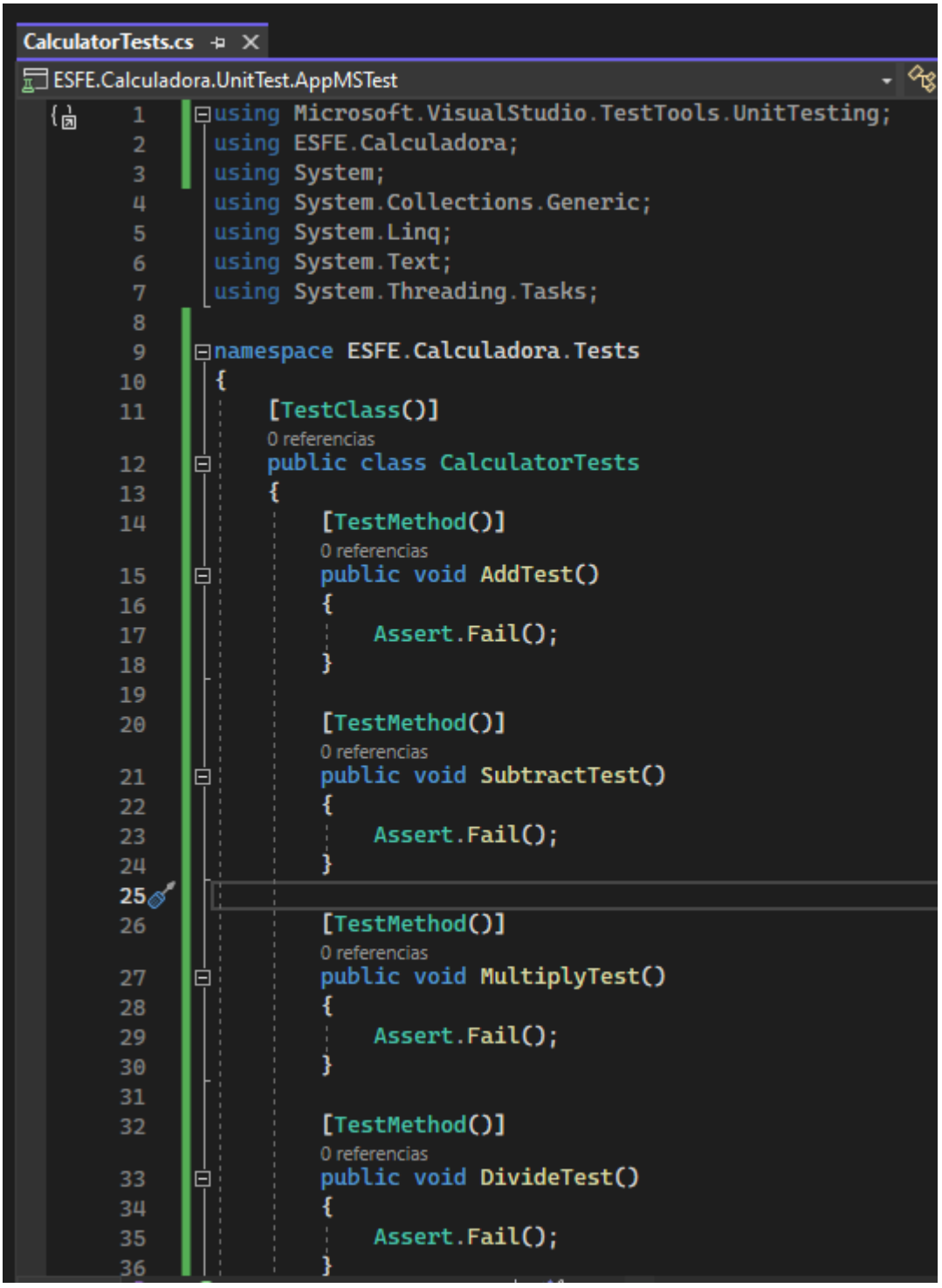
Ahora, vamos a generar automáticamente los métodos que vamos a probar, centrándonos en la clase Calculator..

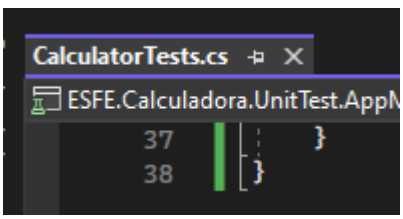



Calculator.cs
ESFE.Calculadora
ESFE.Calculadora.Calculator
namespace ESFE.Calculadora
{
0 referencias
public class Calculator
{
Click derecho
Acciones rápidas y refactorizaciones... Ctrl+.
Cambiar nombre... Ctrl+R, Ctrl+R
Quitar y ordenar usos Ctrl+R, Ctrl+G
Ver la definición sin salir Alt+F12
Ir a definición F12
Ir a base Alt+Inicio
Ir a implementación Ctrl+F12
Buscar todas las referencias Mayús.+F12
Ver jerarquía de llamadas Ctrl+K, Ctrl+T
Origen de valor de seguimiento
Ejecutar pruebas Ctrl+R, T
Depurar pruebas Ctrl+R, Ctrl+T
Mostrar en el Explorador de pruebas Ctrl+E, S
Crear pruebas unitarias
Punto de interrupción
Ejecutar hasta el cursor Ctrl+F10
Forzar ejecución hasta el cursor
Ejecutar en modo interactivo Ctrl+E, Ctrl+E
Fragmento de código
Cortar Ctrl+X
Copiar Ctrl+C
not allowed.");

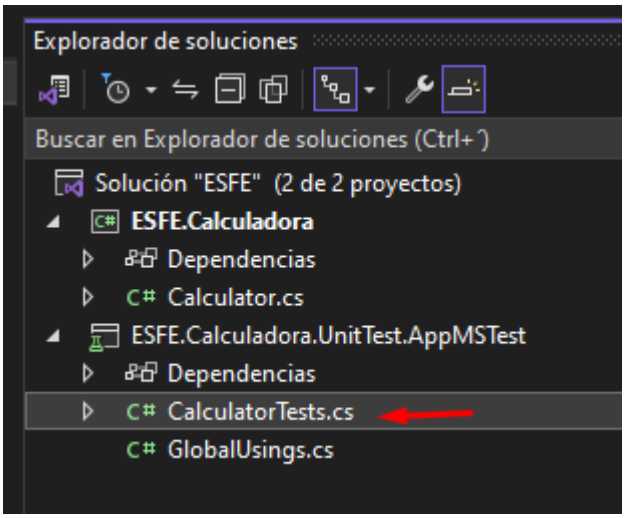

Crear pruebas unitarias
Marco de prueba:
MSTestv2
Obtener extensiones adicionales
Proyecto de prueba:
ESFE.Calculadora.UnitTest.AppMSTest
Formato de prueba para proyecto de prueba:
[Project]Tests
Espacio de nombres:
[Namespace].Tests
Archivo de salida:
<New Test File>
Formato de nombre para clase de prueba:
[Class]Tests
Formato de nombre para método de prueba:
[Method]Test
Código para el método de prueba:
Error de aserción
Aceptar
Cancelar


Explorador de soluciones
Buscar en Explorador de soluciones (Ctrl+ˇ)
Solución "ESFE" (2 de 2 proyectos)
ESFE.Calculadora
Dependencias
Calculator.cs
ESFE.Calculadora.UnitTest.AppMSTest
Dependencias
CalculatorTests.cs
GlobalUsings.cs


CalculatorTests.cs
ESFE.Calculadora.UnitTest.AppMSTest
using Microsoft.VisualStudio.TestTools.UnitTesting;
using ESFE.Calculadora;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace ESFE.Calculadora.Tests
{
[TestClass()]
0 referencias
public class CalculatorTests
{
[TestMethod()]
0 referencias
public void AddTest()
{
Assert.Fail();
}
[TestMethod()]
0 referencias
public void SubtractTest()
{
Assert.Fail();
}
[TestMethod()]
0 referencias
public void MultiplyTest()
{
Assert.Fail();
}
[TestMethod()]
0 referencias
public void DivideTest()
{
Assert.Fail();
}


CalculatorTests.cs
ESFE.Calculadora.UnitTest.AppM
}
}

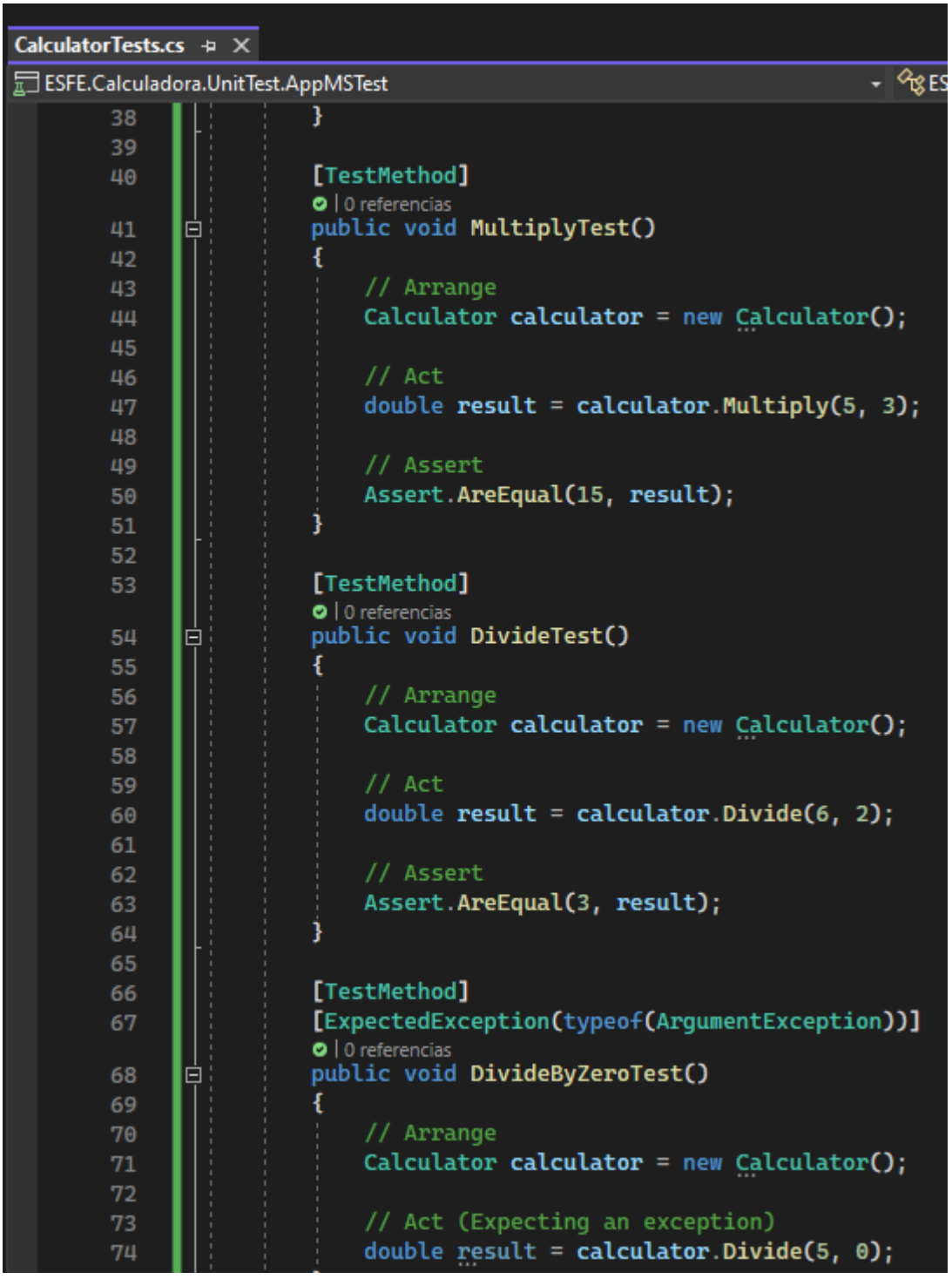
Ahora, procederemos a modificar la clase 'CalculatorTests' para agregar el código necesario y realizar las pruebas unitarias en nuestros métodos de la clase 'Calculator'.

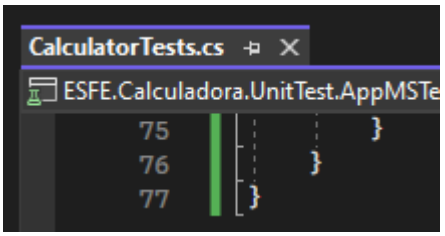


CalculatorTests.cs

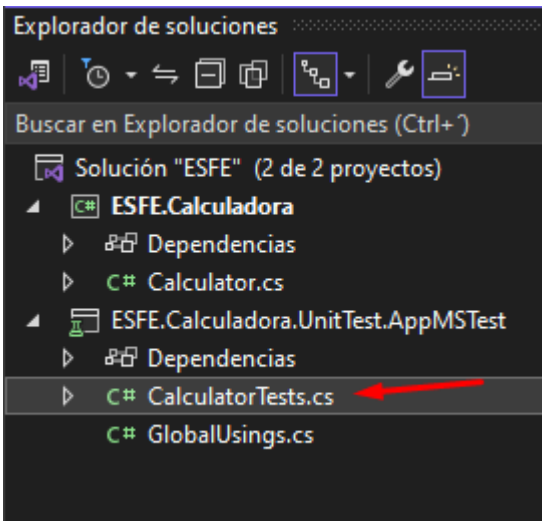
ESFE.Calculadora.UnitTest.AppMSTest

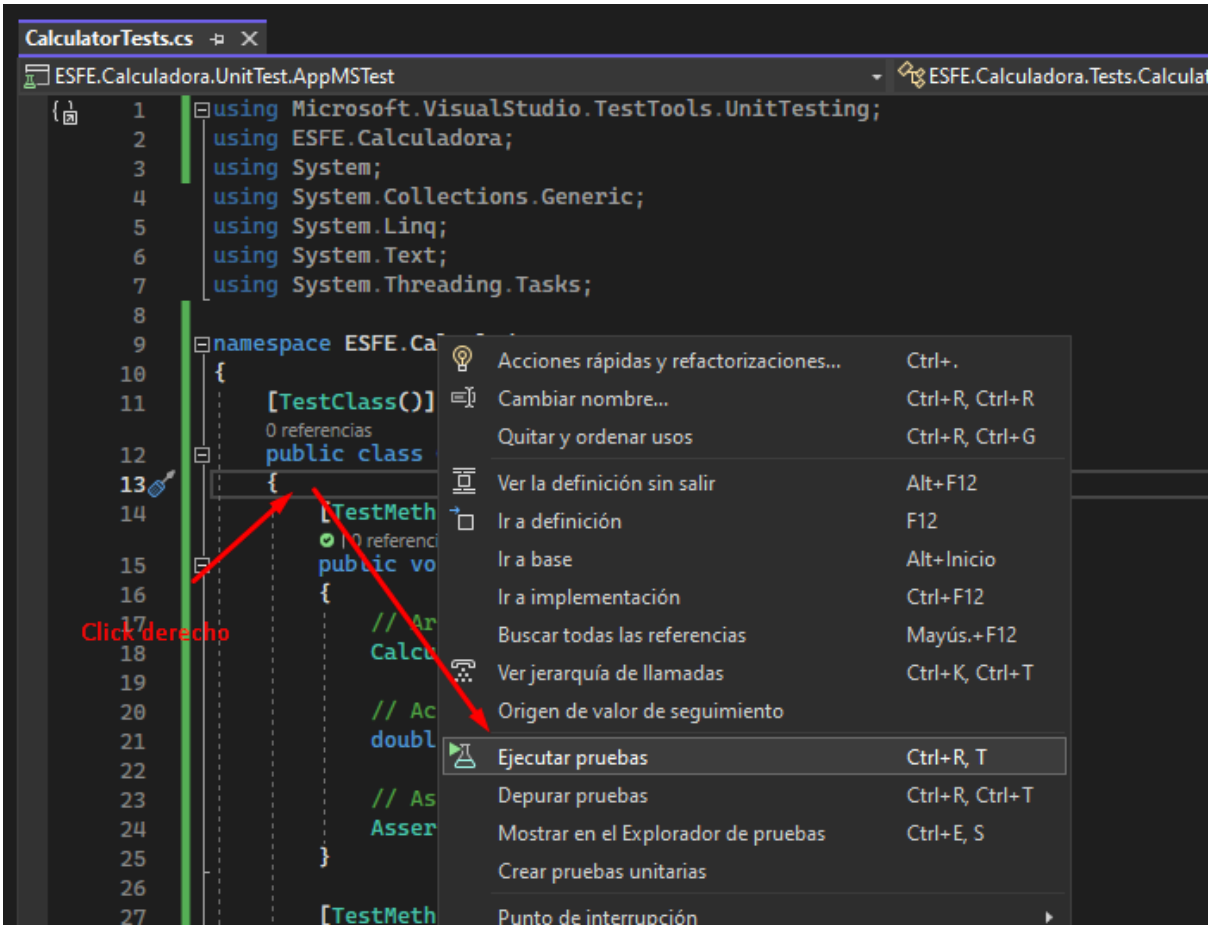
```
1  using Microsoft.VisualStudio.TestTools.UnitTesting;
2  using ESFE.Calculadora;
3  using System;
4  using System.Collections.Generic;
5  using System.Linq;
6  using System.Text;
7  using System.Threading.Tasks;
8
9  namespace ESFE.Calculadora.Tests
10 {
11     [TestClass()]
       0 referencias
12     public class CalculatorTests
13     {
14         [TestMethod]
           0 referencias
15         public void AddTest()
16         {
17             // Arrange
18             Calculator calculator = new Calculator();
19
20             // Act
21             double result = calculator.Add(5, 3);
22
23             // Assert
24             Assert.AreEqual(8, result);
25         }
26
27         [TestMethod]
           0 referencias
28         public void SubtractTest()
29         {
30             // Arrange
31             Calculator calculator = new Calculator();
32
33             // Act
34             double result = calculator.Subtract(5, 3);
35
36             // Assert
37             Assert.AreEqual(2, result);
```



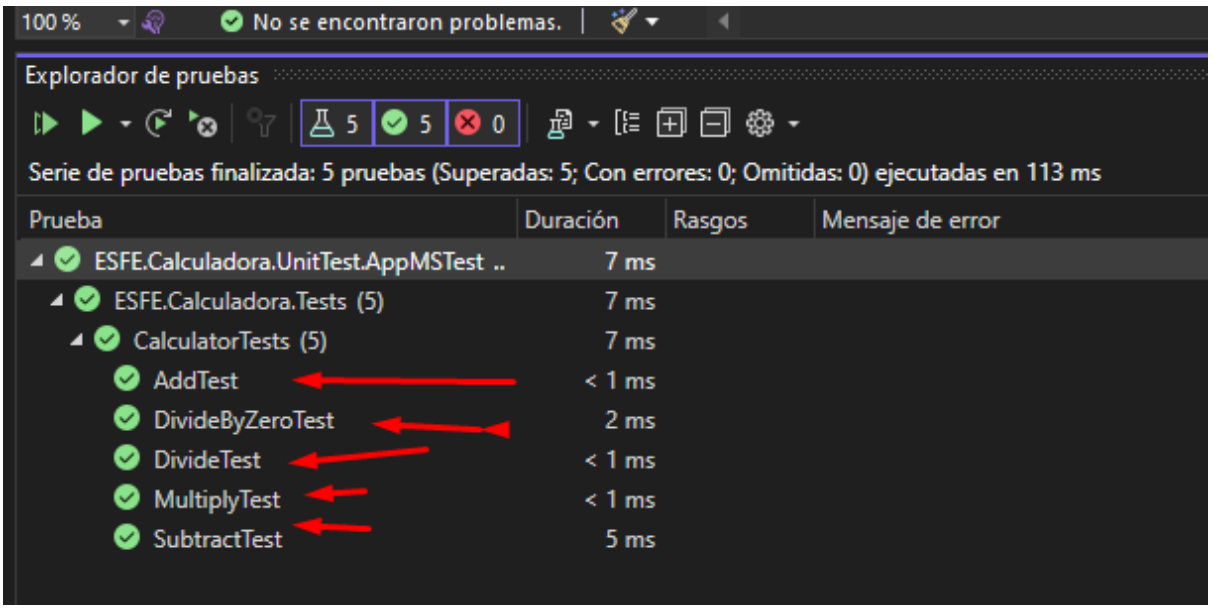


Lo siguiente que haremos es ejecutar la prueba unitaria de la clase 'CalculatorTests'.

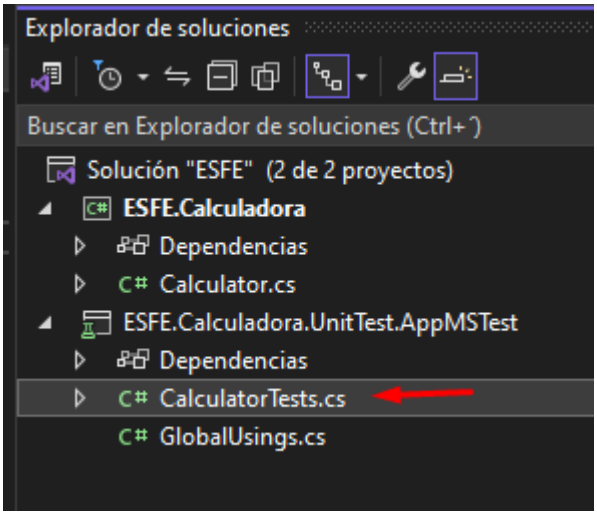


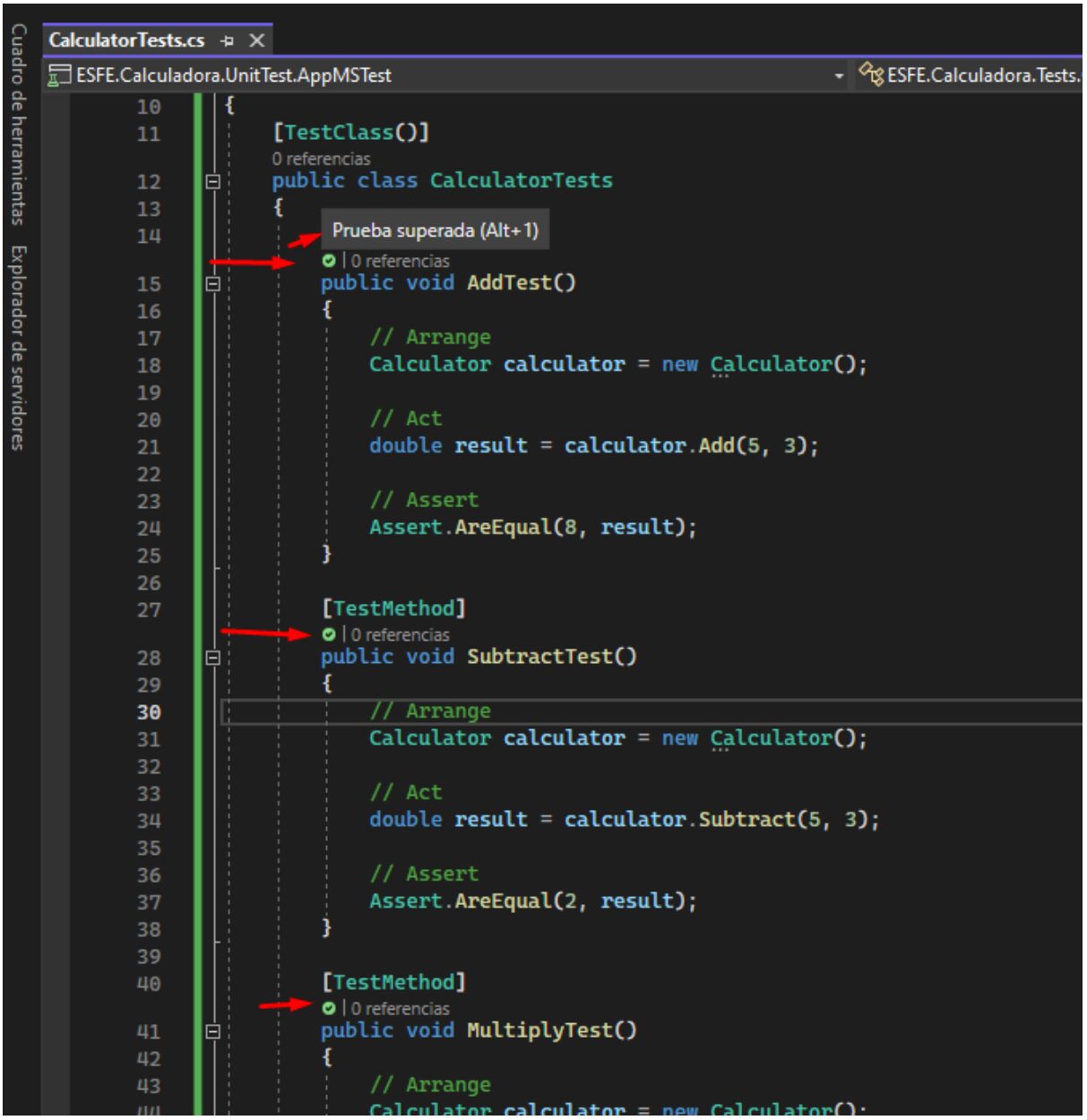


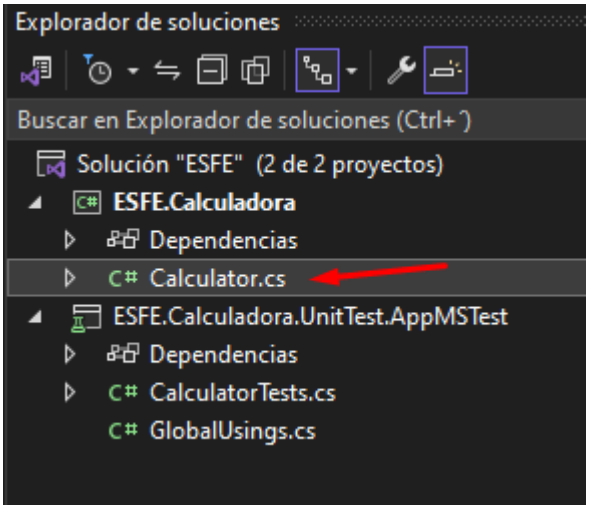
Si aparecen con el marcador de verificación verde, significa que las pruebas fueron exitosas.

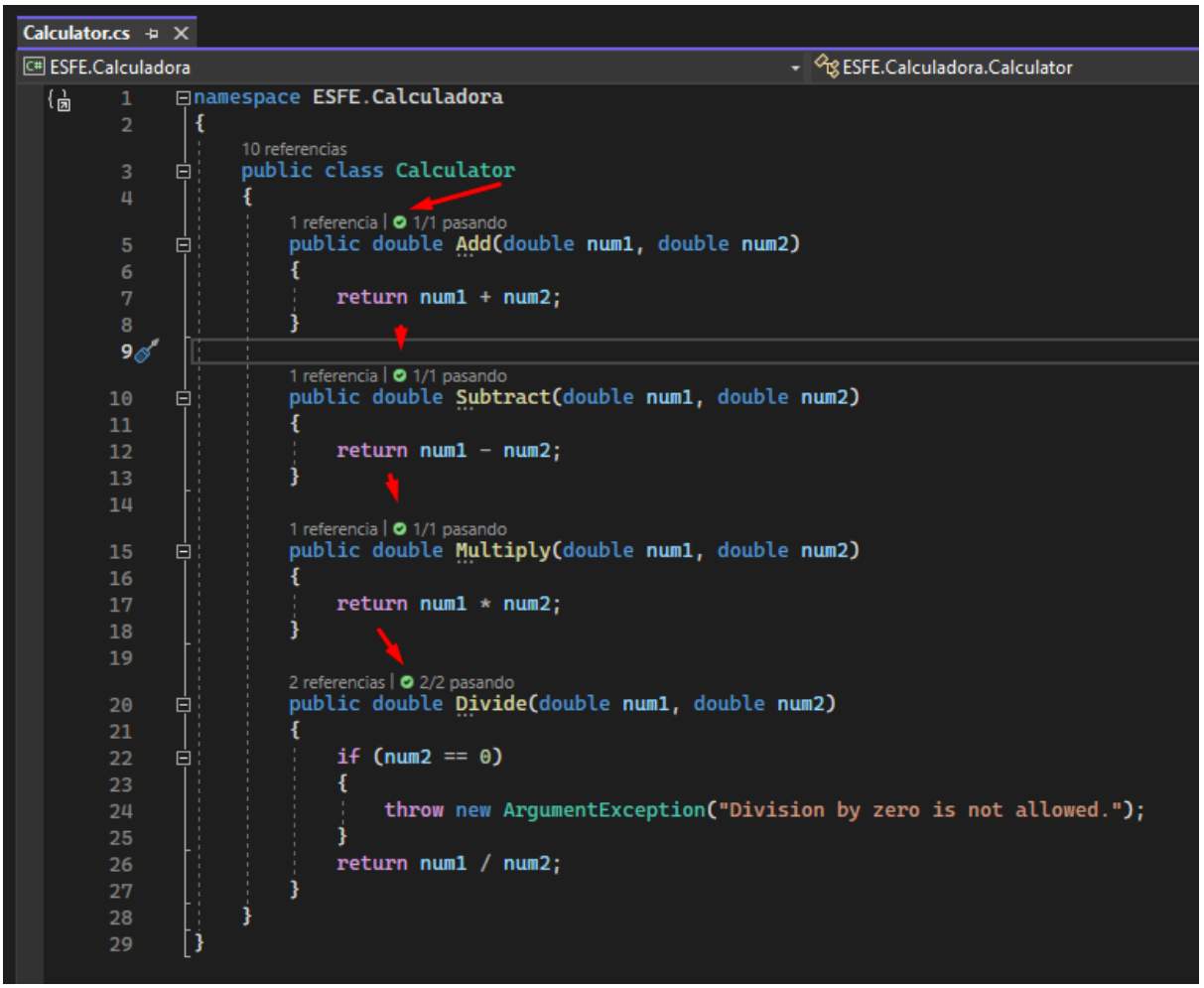


Además, en Visual Studio, los métodos que han sido probados correctamente están marcados con un símbolo de verificación verde tanto en el proyecto que ejecuta las pruebas unitarias como en el proyecto donde se encuentra el método original que se está probando.



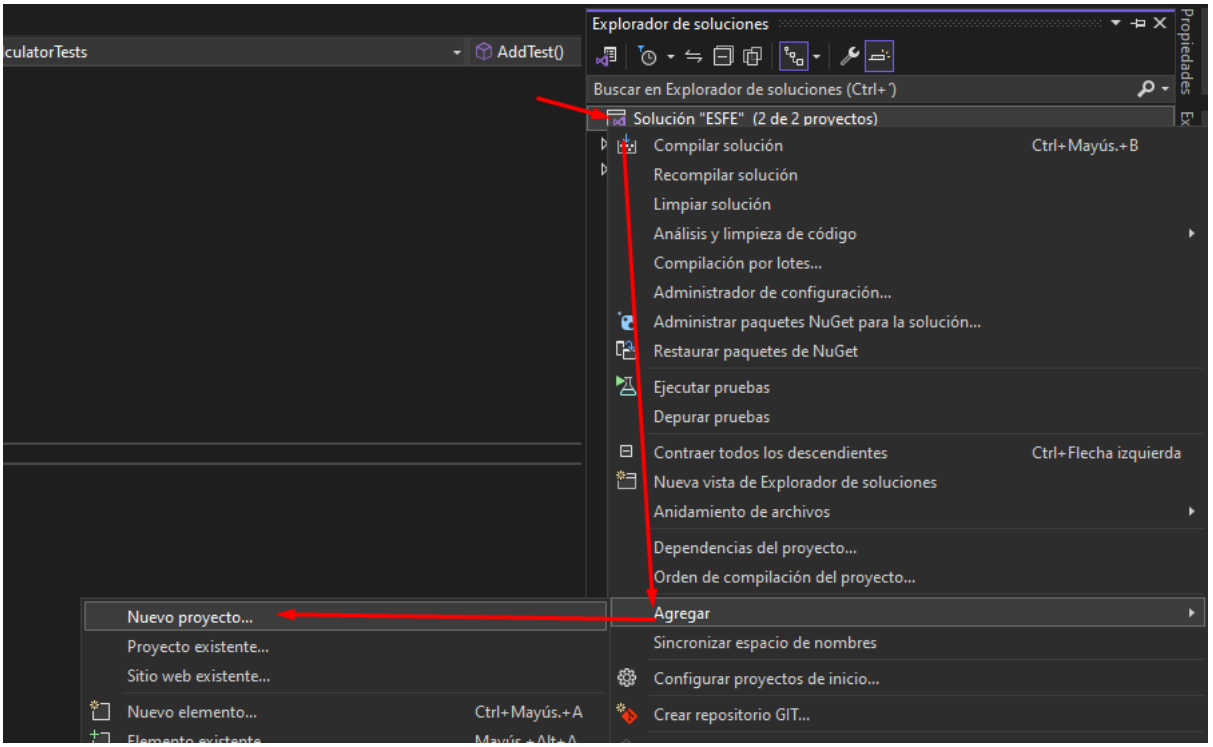

CalculatorTests.cs
ESFE.Calculadora.UnitTest.AppMSTest
ESFE.Calculadora.Tests.
Cuadro de herramientas
Explorador de servidores
Prueba superada (Alt+1)
0 referencias
[TestClass()]
public class CalculatorTests
public void AddTest()
// Arrange
Calculator calculator = new Calculator();
// Act
double result = calculator.Add(5, 3);
// Assert
Assert.AreEqual(8, result);
[TestMethod]
public void SubtractTest()
double result = calculator.Subtract(5, 3);
Assert.AreEqual(2, result);
public void MultiplyTest()

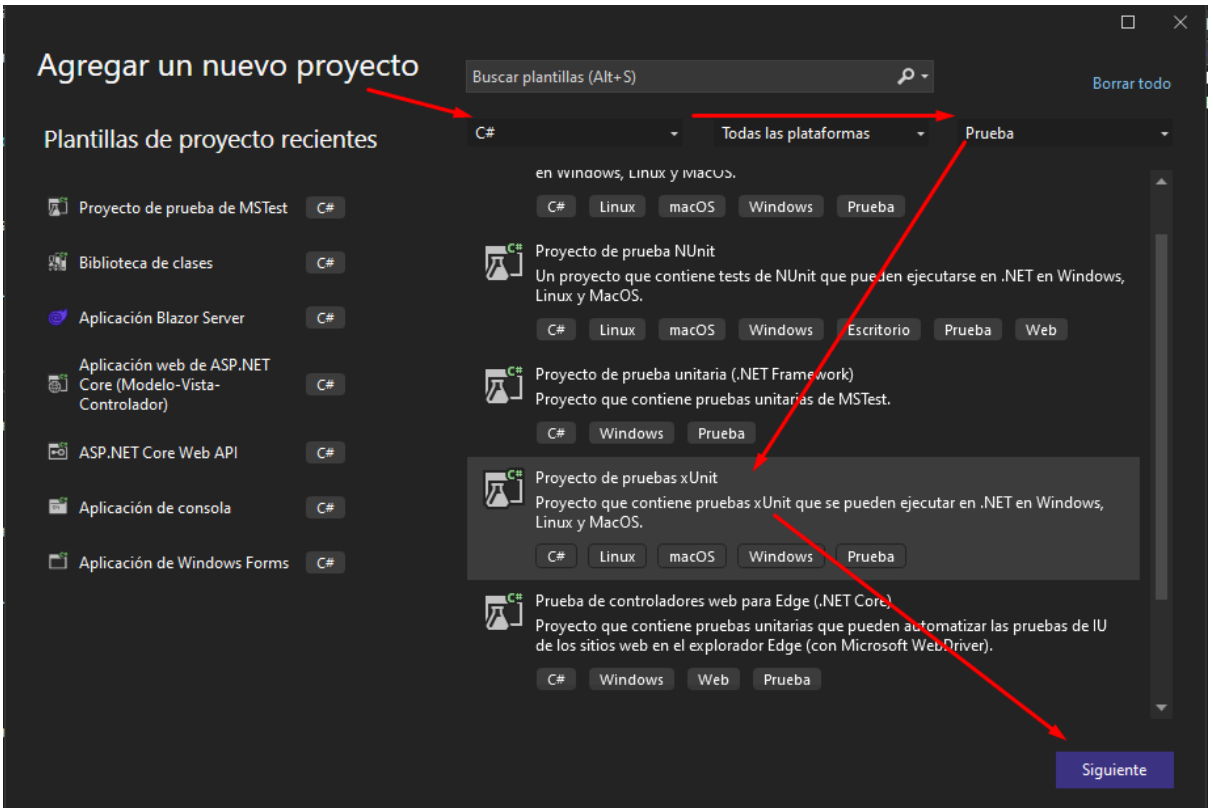

Explorador de soluciones
Buscar en Explorador de soluciones (Ctrl+ˆ)
Solución "ESFE" (2 de 2 proyectos)
ESFE.Calculadora
Dependencias
Calculator.cs
ESFE.Calculadora.UnitTest.AppMSTest
Dependencias
CalculatorTests.cs
GlobalUsings.cs

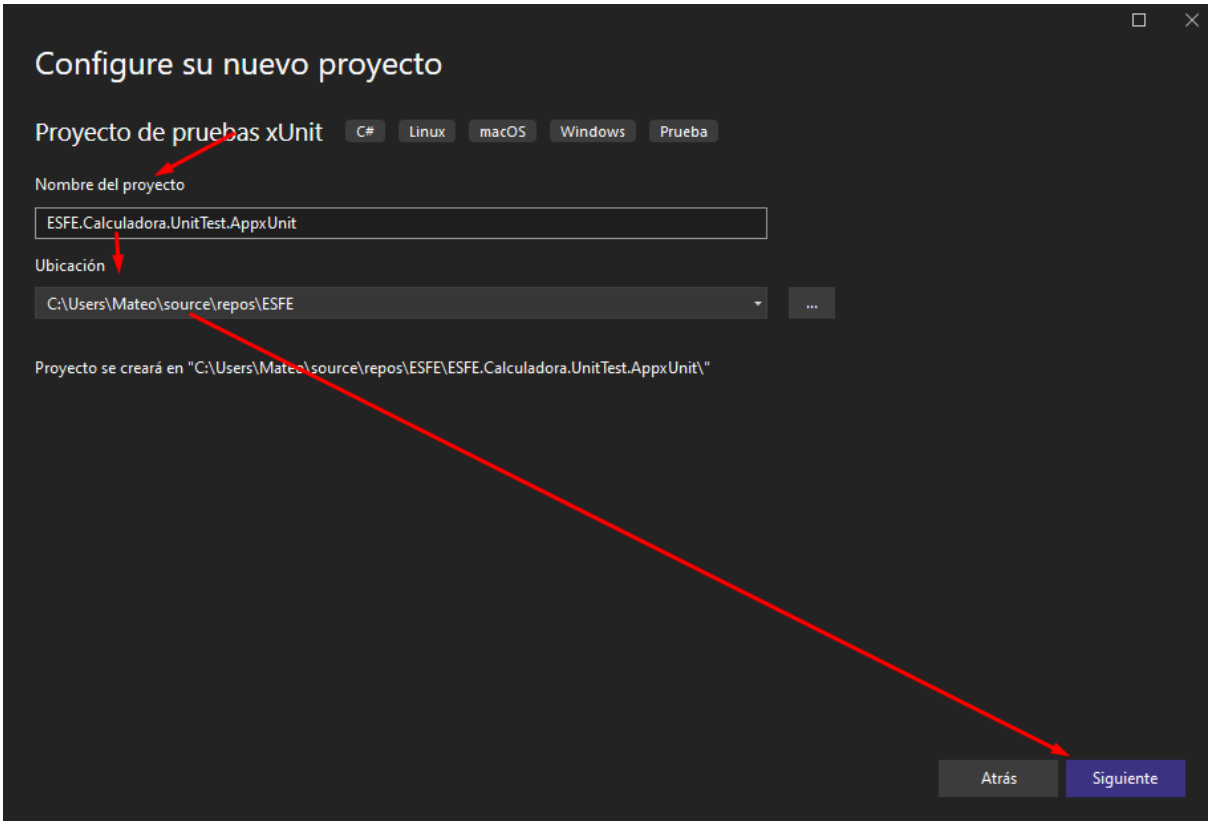


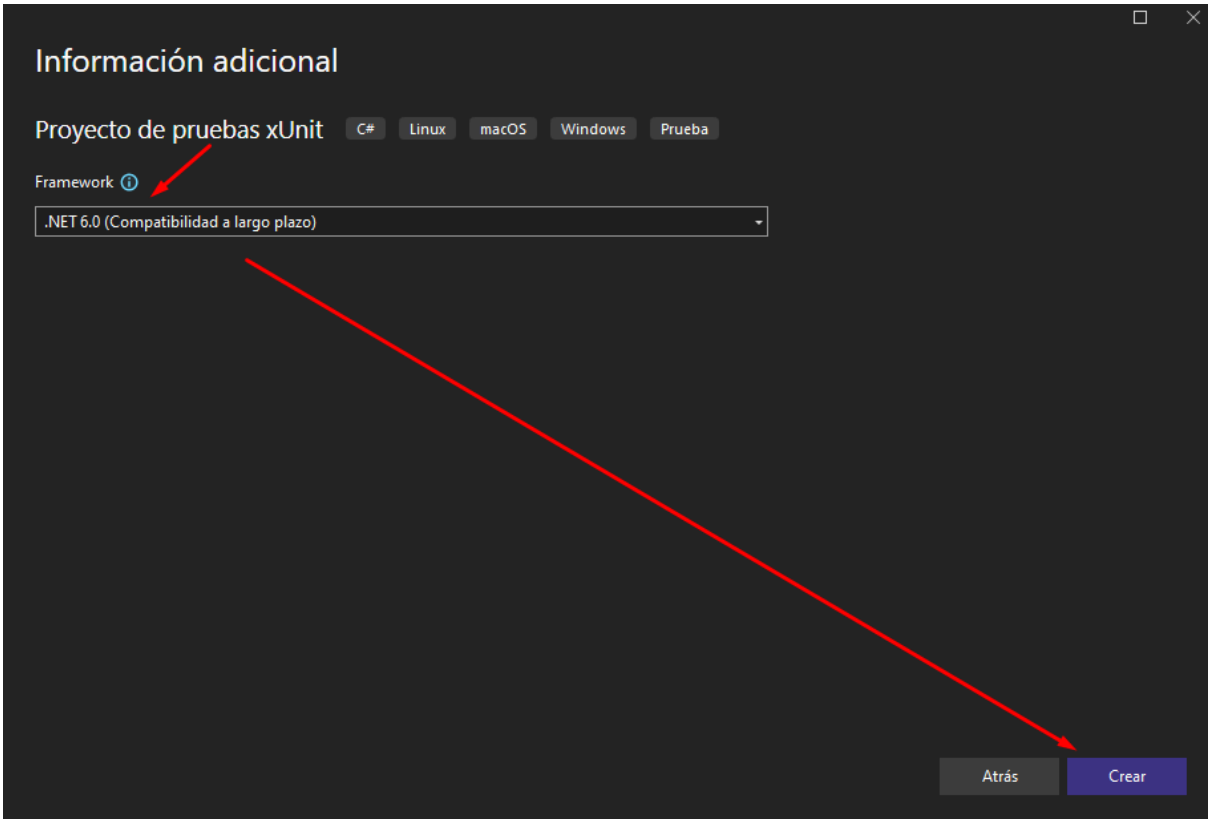
Ahora, aprendamos cómo crear un proyecto de pruebas unitarias utilizando xUnit

Comenzaremos creando el proyecto dentro de la misma solución actual

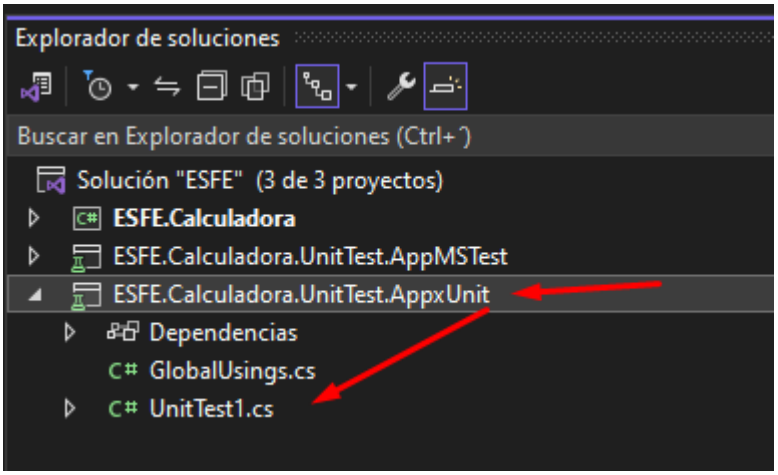

culatorTests
AddTest()
Explorador de soluciones
Buscar en Explorador de soluciones (Ctrl+ ')
Solución "ESFE" (2 de 2 proyectos)
Compilar solución Ctrl+Mayús.+B
Recompilar solución
Limpiar solución
Análisis y limpieza de código
Compilación por lotes...
Administrador de configuración...
Administrar paquetes NuGet para la solución...
Restaurar paquetes de NuGet
Ejecutar pruebas
Depurar pruebas
Contraer todos los descendientes Ctrl+Flecha izquierda
Nueva vista de Explorador de soluciones
Anidamiento de archivos
Dependencias del proyecto...
Orden de compilación del proyecto...
Nuevo proyecto...
Proyecto existente...
Sitio web existente...
Nuevo elemento... Ctrl+Mayús.+A
Agregar
Sincronizar espacio de nombres
Configurar proyectos de inicio...
Crear repositorio GIT...
Propiedades

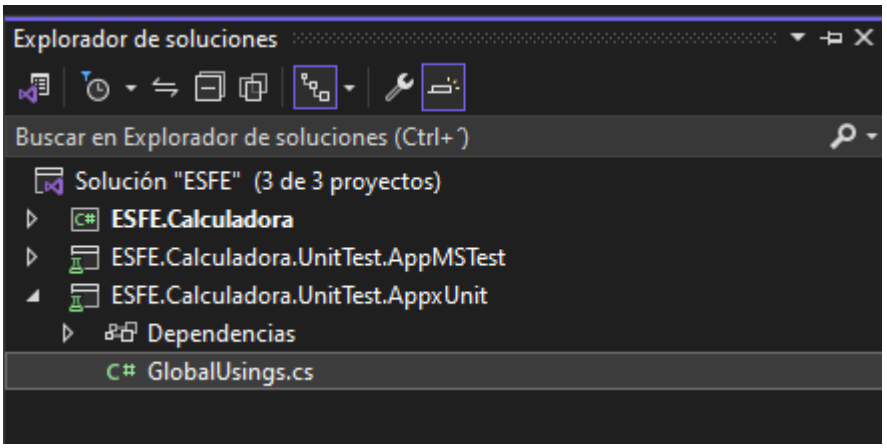

Agregar un nuevo proyecto
Buscar plantillas (Alt+S)
Borrar todo
Plantillas de proyecto recientes
C#
Todas las plataformas
Prueba
Proyecto de prueba de MSTest C#
Biblioteca de clases C#
Aplicación Blazor Server C#
Aplicación web de ASP.NET Core (Modelo-Vista-Controlador) C#
ASP.NET Core Web API C#
Aplicación de consola C#
Aplicación de Windows Forms C#
C# Linux macOS Windows Prueba
Proyecto de prueba NUnit
Un proyecto que contiene tests de NUnit que pueden ejecutarse en .NET en Windows, Linux y MacOS.
C# Linux macOS Windows Escritorio Prueba Web
Proyecto de prueba unitaria (.NET Framework)
Proyecto que contiene pruebas unitarias de MSTest.
C# Windows Prueba
Proyecto de pruebas xUnit
Proyecto que contiene pruebas xUnit que se pueden ejecutar en .NET en Windows, Linux y MacOS.
C# Linux macOS Windows Prueba
Prueba de controladores web para Edge (.NET Core)
Proyecto que contiene pruebas unitarias que pueden automatizar las pruebas de IU de los sitios web en el explorador Edge (con Microsoft WebDriver).
C# Windows Web Prueba
Siguiente



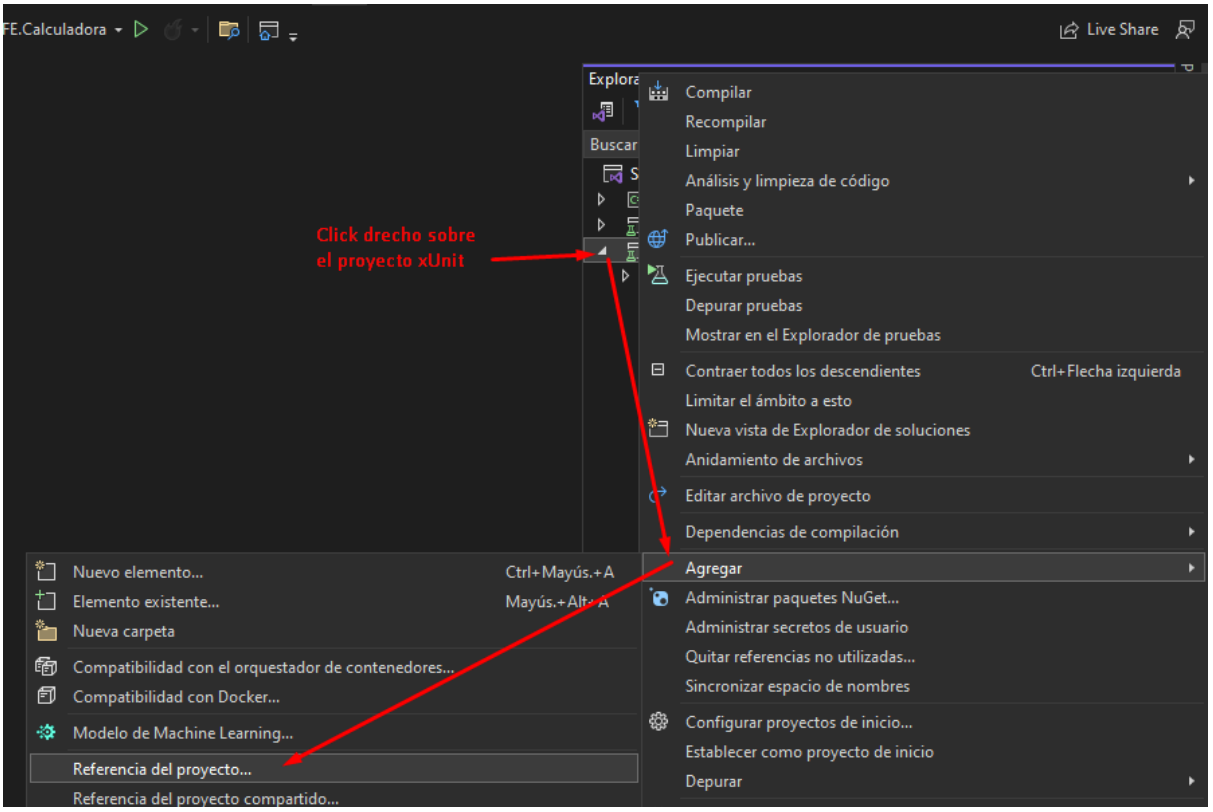


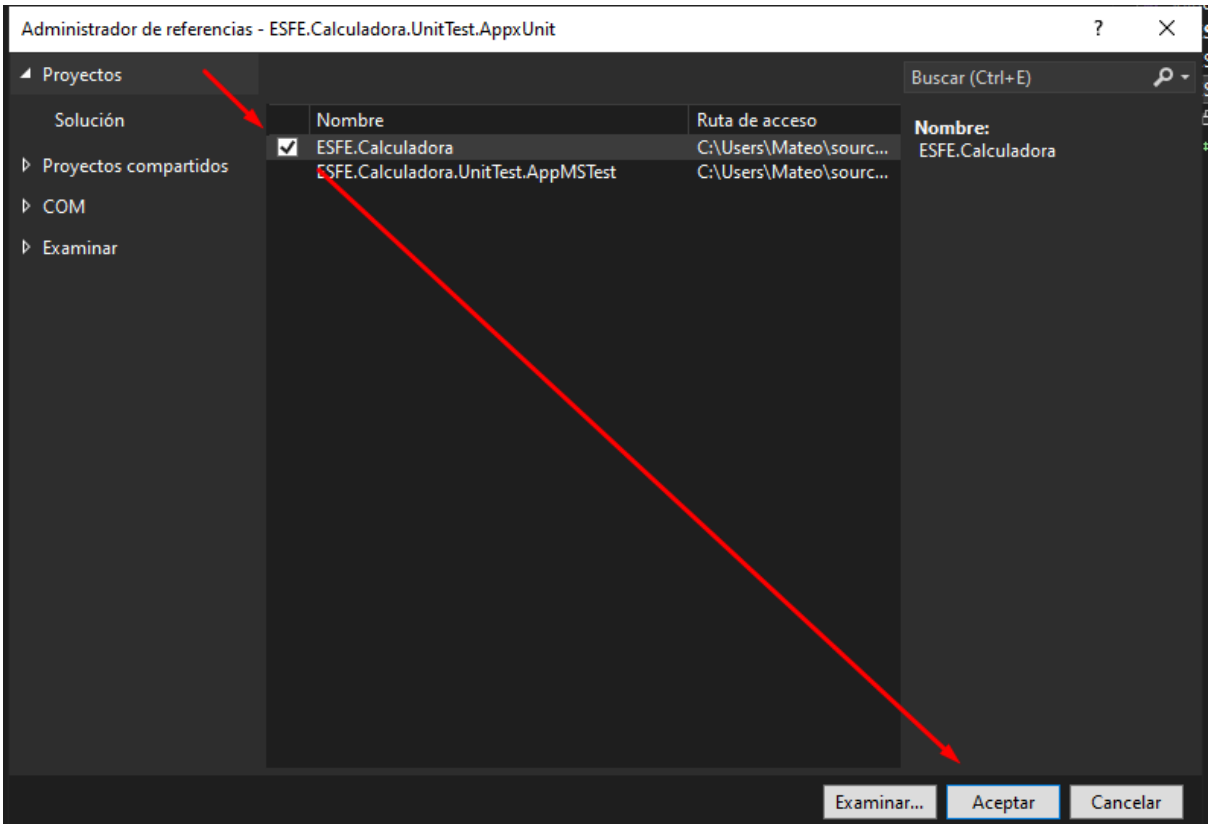
Vamos a eliminar la clase 'UnitTest1.cs' que se creó por defecto.



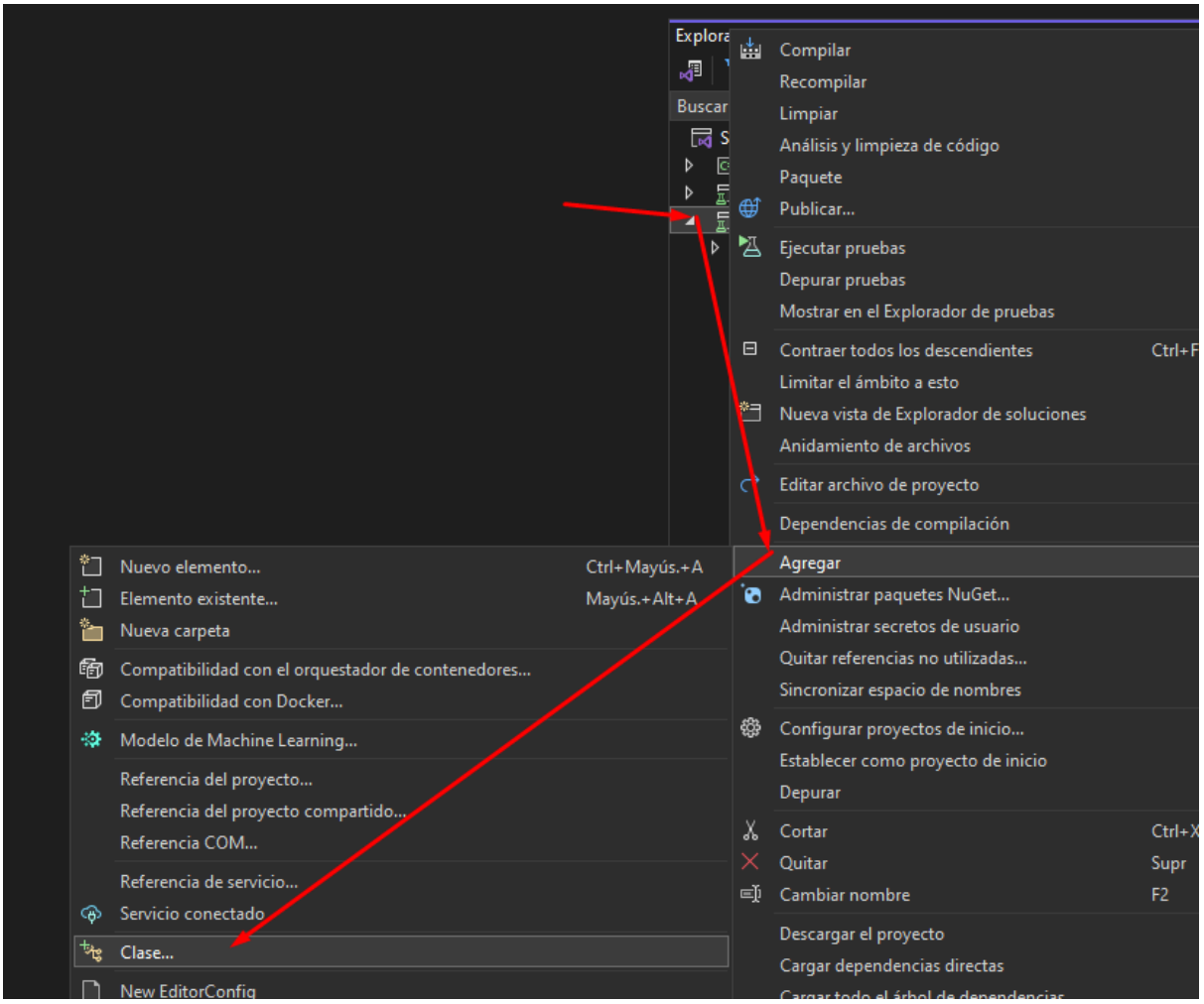


Lo siguiente es agregar las referencias del proyecto al que le realizaremos las pruebas unitarias, en nuestro caso, será el proyecto 'ESFE.Calculadora'.

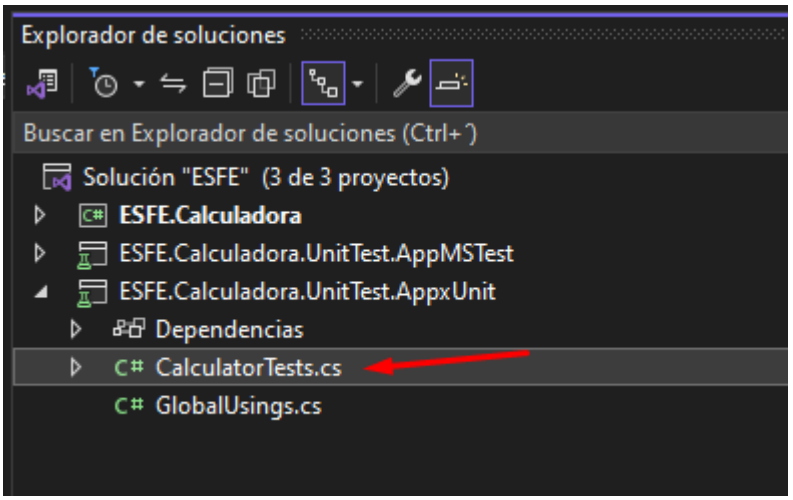


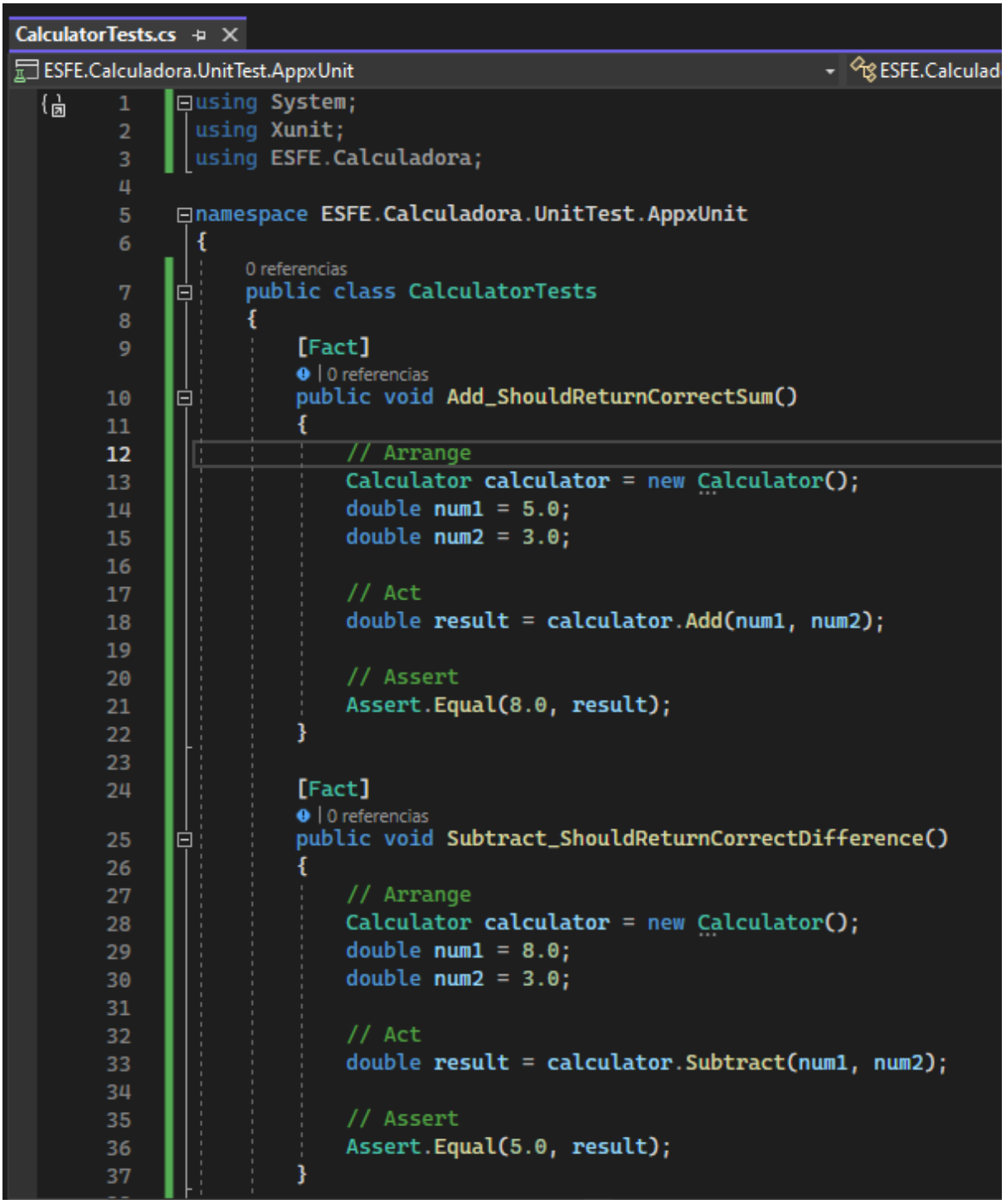


Ahora, vamos a crear la clase 'CalculatorTests' dentro del proyecto 'ESFE.Calculadora.UnitTest.AppxUnit' donde programaremos los métodos para realizar las pruebas unitarias en la clase 'Calculator'.



Vamos a modificar la clase 'CalculatorTest' para programar nuestras pruebas unitarias de los métodos de la clase 'Calculator'.




CalculatorTests.cs
ESFE.Calculadora.UnitTest.AppxUnit
ESFE.Calculad
using System;
using Xunit;
using ESFE.Calculadora;
namespace ESFE.Calculadora.UnitTest.AppxUnit
{
0 referencias
public class CalculatorTests
{
[Fact]
0 referencias
public void Add_ShouldReturnCorrectSum()
{
// Arrange
Calculator calculator = new Calculator();
double num1 = 5.0;
double num2 = 3.0;
// Act
double result = calculator.Add(num1, num2);
// Assert
Assert.Equal(8.0, result);
}
[Fact]
0 referencias
public void Subtract_ShouldReturnCorrectDifference()
{
// Arrange
Calculator calculator = new Calculator();
double num1 = 8.0;
double num2 = 3.0;
// Act
double result = calculator.Subtract(num1, num2);
// Assert
Assert.Equal(5.0, result);
}

CalculatorTests.cs

ESFE.Calculadora.UnitTest.AppxUnit

```
38
39          [Fact]
            0 referencias
40          public void Multiply_ShouldReturnCorrectProduct()
41          {
42              // Arrange
43              Calculator calculator = new Calculator();
44              double num1 = 4.0;
45              double num2 = 2.0;
46
47              // Act
48              double result = calculator.Multiply(num1, num2);
49
50              // Assert
51              Assert.Equal(8.0, result);
52          }
53
54          [Fact]
            0 referencias
55          public void Divide_ShouldReturnCorrectQuotient()
56          {
57              // Arrange
58              Calculator calculator = new Calculator();
59              double num1 = 10.0;
60              double num2 = 2.0;
61
62              // Act
63              double result = calculator.Divide(num1, num2);
64
65              // Assert
66              Assert.Equal(5.0, result);
67          }
68
69          [Fact]
            0 referencias
70          public void Divide_ByZero_ShouldThrowException()
71          {
72              // Arrange
73              Calculator calculator = new Calculator();
74              double num1 = 5.0;
```

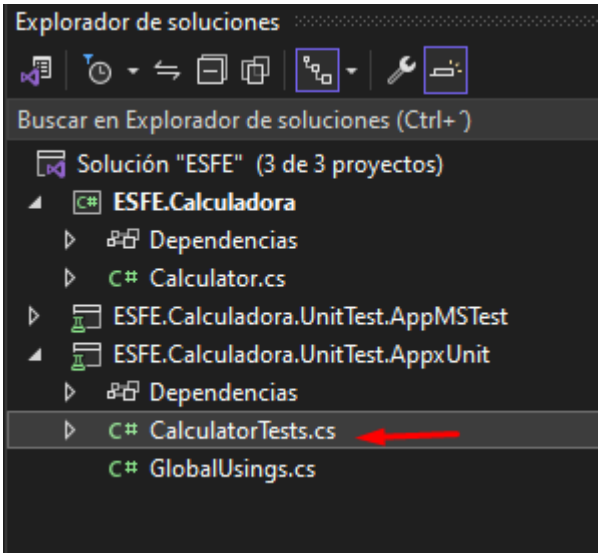
CalculatorTests.cs

ESFE.Calculadora.UnitTest.AppxUnit

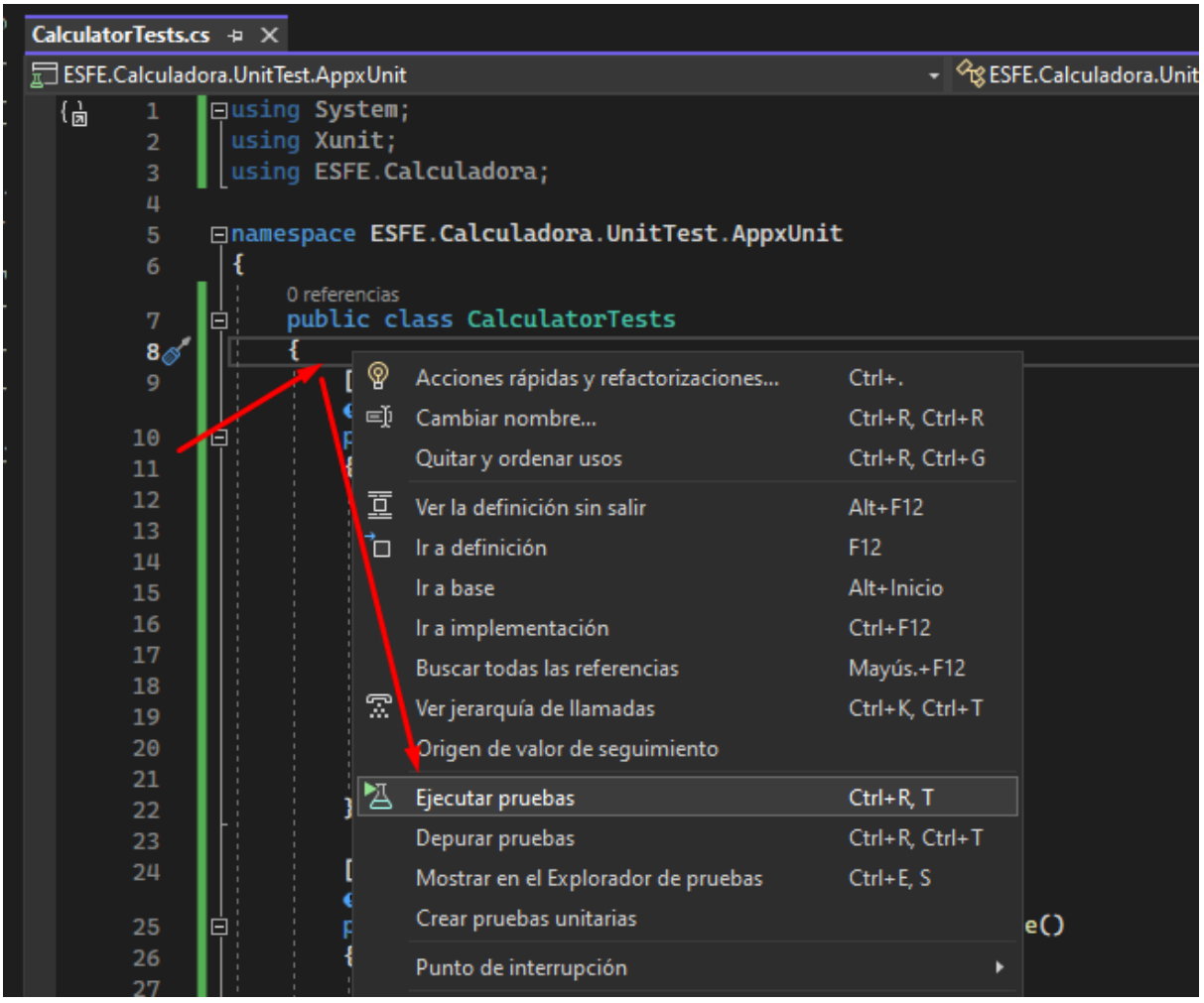
ESFE.Calculadora.UnitTest.AppxUnit.Calcu

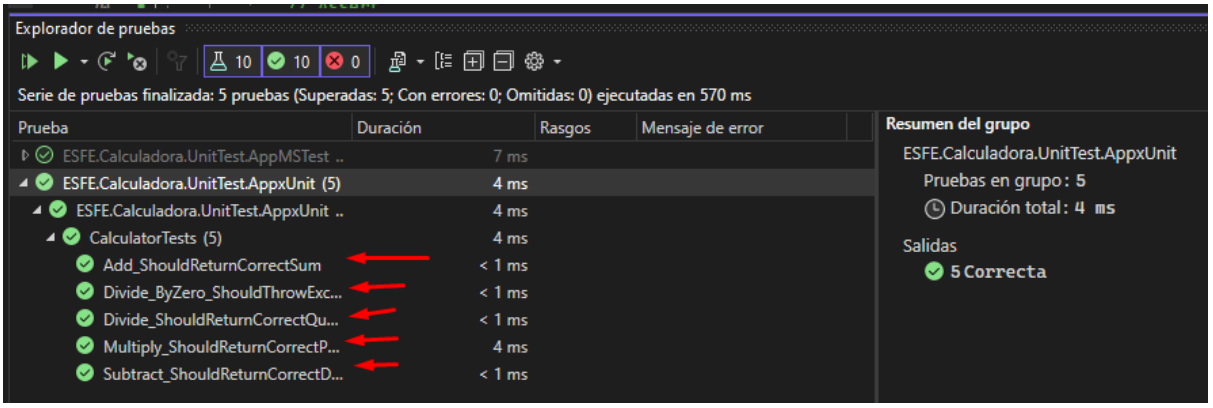
```
75              double num2 = 0.0;
76
77              // Act and Assert
78              Assert.Throws<ArgumentException>(() => calculator.Divide(num1, num2));
79          }
80      }
81  }
82
```

Ahora vamos a ejecutar nuestras pruebas unitarias.

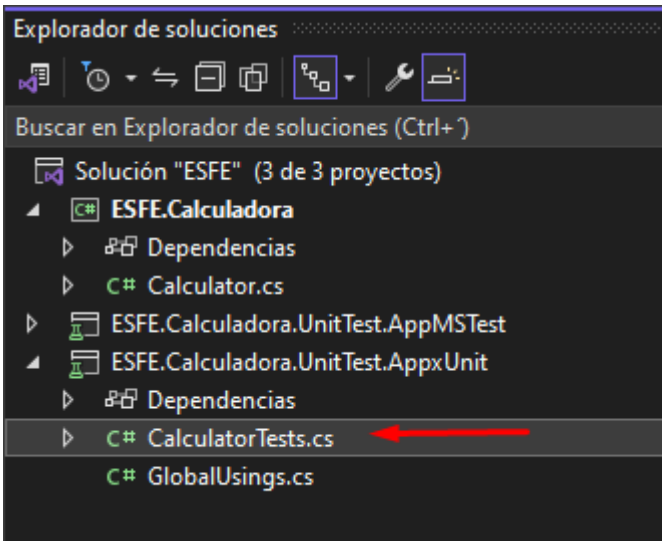


Hacemos clic derecho en el punto de inicio del código de la clase para ejecutar todos los métodos que contiene.

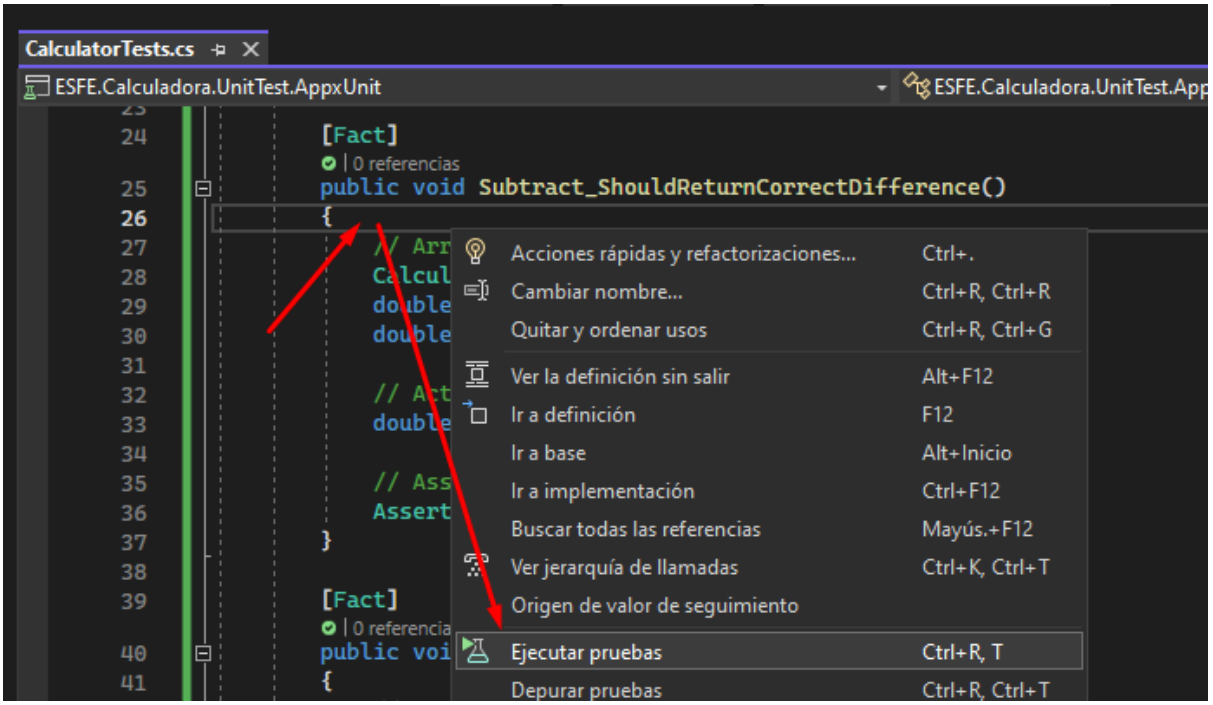


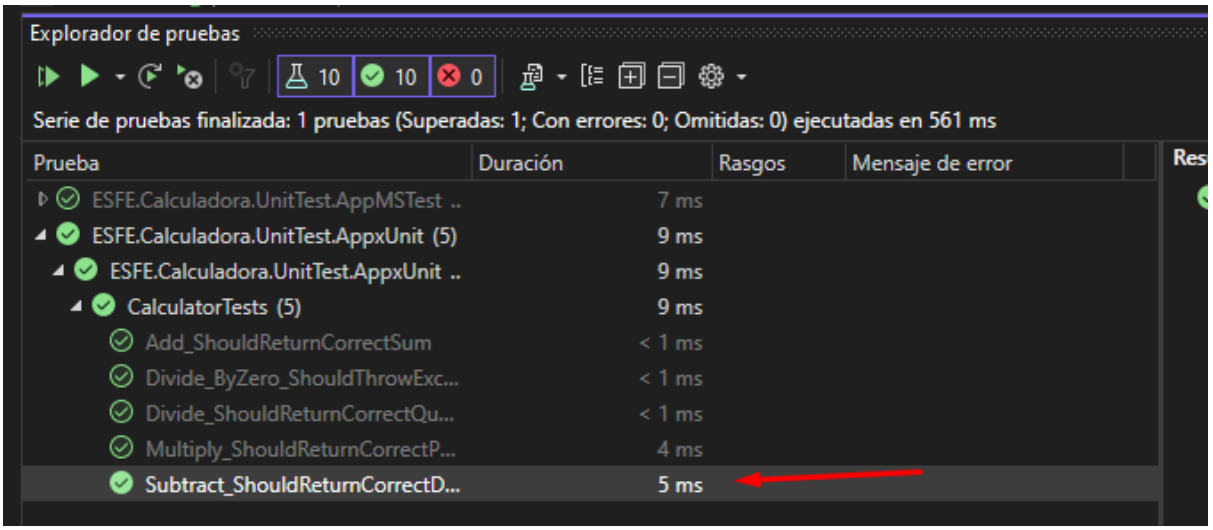


También podemos ejecutar la prueba unitaria de un método en específico.



Hacemos clic derecho en el método para ejecutar esa prueba unitaria específica.





Repositorio en github: https://github.com/ESFEAlumnos2023/unittest

Realizar los siguientes ejercicios prácticos

Indicaciones:

- El nombre de la solución debe de contener sus iniciales de sus nombres y apellidos ejemplo: `Marvin Antonio Barrera trigueros` el cual quedaría mi solución con el siguiente nombre MABT
- Los nombre de los proyectos debe contener al inicio el nombre de la solución y después del punto el nombre aque se refiere el ejercicio ejemplo: MABT.Temperatura para el proyecto de librería de clase y MABT.Temperatura.UnitTestMSTest para el proyecto para realizar las pruebas unitarias, eso dependerá del tipo de proyecto que se solicite hacer las pruebas unitarias en los ejercicios
- Al momento de iniciar los proyecto y la solución iniciar también el repositorio git de manera local y como vaya avanzando ir realizando los commit como historial de su trabajo si es necesario subir el repositorio a github para su revisión.
- Documentar los casos de prueba en azure devops de los dos ejercicios

1. Vamos a realizar las pruebas unitarias utilizando MSTest para un Convertidor de Temperatura que incluye la siguiente clase y los métodos que vamos a probar.

```
public class TemperatureConverter
{
    public double CelsiusToFahrenheit(double celsius)
    {
        // Implementa la conversión de Celsius a Fahrenheit
    }

    public double FahrenheitToCelsius(double fahrenheit)
    {
        // Implementa la conversión de Fahrenheit a Celsius
    }
}
```

2. Vamos a realizar pruebas unitarias utilizando xUnit para realizar manipulaciones simples en cadenas de texto. La clase a probar incluye los siguientes métodos que vamos a testear.

```
public class StringManipulator
{
    public string ReverseString(string input)
    {
        // Implementa la inversión de la cadena
    }

    public string RemoveSpaces(string input)
    {
        // Implementa la eliminación de espacios en blanco
    }
}
```